VIT VELLORE · SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

BCSE307L – Compiler Design

FINAL PROJECT REPORT · REVIEW 3

PYTHIA and PYTHIA Studio

A verified source-to-source accelerator that converts the slow parts of a Python project to C++17, checks every conversion against CPython, and splices it back in

Item	Details
Course	BCSE307L – Compiler Design
Project ID and title	A15 – Typed Source-to-Source Transpiler
Team number	Team 2
Review	Review 3 (final), October 2026
Faculty	_____
Repository state	branch `main`, commit `95bbb1b` (143 commits)

Team member	Register number	Role
Vishal Vivek (team lead)	24BCE2377	Front end, integration, repository, Studio application
Rujuta Mangesh Kulkarni	24BDS0459	Type checker, semantic analysis, dictionary support, negative corpus
Koliparthy Venkata Jahnavi	24BDS0132	Lowering, divergence catalogue, runtime, guard elision, PySet
Aaryan Gupta	24BDS0134	C++ emitter, C11 to C++17 migration, differential oracle, CI, corpus

Declaration. We declare that this report and the project it describes are the work of the team named above. Where we reused open-source software, documentation or AI coding assistants, we have said so in Section 14. Every number in this report comes from a command we ran on the machine described in Section 5; Appendix F lists each claim with its command, result and date.

Signatures: _____ _____ _____ _____

Contents

1. Abstract 3
2. Introduction and motivation 3
3. Problem statement, objectives, scope and limitations 4
4. Background and related work 6
5. Requirements and technology stack 8
6. System design 10
7. Algorithms and methodology 13
8. Implementation details 18
9. Testing strategy and test cases 19
10. Results and evaluation 20
11. Error handling, limitations and known issues 23
12. Member-wise contribution 25
13. Conclusion and future work 26
14. References 27
Appendix A. User manual 29
Appendix B. Installation guide 30
Appendix C. Sample runs 31
Appendix D. Complete test table 35
Appendix E. Grammar specification and API specification 39
Appendix F. Evidence register 41
Appendix G. Contribution log 43
Appendix H. User-interface screenshots and additional diagrams 44

1. Abstract

Python is easy to write and slow on loop-heavy code, and the usual remedies either cost a second code base or give no evidence that the fast version behaves like the original. PYTHIA is a source-to-source compiler and desktop application that converts the hot functions of an ordinary Python project to C++17, checks every conversion against CPython on the program's own recorded calls, and splices the survivors back in behind a one-line decorator. The compiler core (off-side-rule lexer, Pratt parser, bidirectional type checker, lowering with a fourteen-class Semantic Fidelity Report, interval-based guard elision, C++17 emitter) is extended by an in-house type inference engine for untyped code that uses no language model, a tracer and hotspot ranker, pybind11 library mode, and a replay-based verifier. PYTHIA Studio shows every stage and installs as a 88.8 MiB Windows package. On the machine described in Section 5 the differential oracle agrees with CPython on 78 of 78 programs and rejects 88 of 88 invalid ones; 91 of 91 untyped everyday programs convert and match without any LLM. On ten fixed kernels the compiled binaries are 1.5 to 30 times faster than CPython on nine (median over all ten 11x) with identical output, and about 29 times slower on one dictionary-heavy kernel. Through the full Studio pipeline on a small public repository (dayyass/project-euler), a hot function ran 21.3x faster, verified on 500 of 500 recorded calls, and the whole program went from 8.9 s to 0.8 s with identical output. The limits are stated: 64-bit integers, ASCII strings, no C-extension libraries, and verification that is differential testing rather than proof. The WebAssembly back end promised at Review 2 was dropped in favour of the Studio pipeline.

2. Introduction and motivation

2.1 Context

Python is the default language for scripting, teaching and prototyping, and its speed on loop-heavy, numeric and text-processing code is a well-known cost: the CPython interpreter evaluates every operation on heap-allocated, reference-counted objects, so an integer addition is a pointer chase, a type check and an allocation rather than one machine instruction. Developers who hit this wall have four options today: rewrite the hot code in C or C++, port it to an annotated dialect such as Cython, adopt a JIT such as Numba or PyPy, or ask a language model to translate it. Each option moves the problem rather than solving it. A rewrite costs a second codebase; a dialect costs a new syntax; a JIT constrains the types or the interpreter; and a translation from a model comes with no evidence that it behaves like the original.

2.2 The need

What a working developer actually has is an existing, untyped Python project and a profiler output that says three functions take most of the time. What they want is for those three functions to be fast without rewriting the project, without trusting a black box, and without breaking behaviour. That is a compiler problem in the strict sense: it needs a front end (to read the code), a type system (to know what the code means), an intermediate form (to make Python's implicit behaviour explicit), a back end (to produce native code), and a correctness argument (to say why the output can replace the input). PYTHIA is built around that last point: the conversion is accepted only if it reproduces the original's behaviour on real calls.

2.3 Beneficiaries

- **Developers with an existing Python code base** who want measured speedups on their hot spots, with a diff they can review and a one-line way to undo it.
- **Teams porting or auditing legacy Python**, who get a source-positioned list of every place where Python and C++ semantics could differ (the Semantic Fidelity Report).
- **Students and instructors of compiler design**, because every classical phase is present, small enough to read, and visible in the Studio application.

2.4 Relevance to compiler design

The project exercises each phase of the course: lexical analysis with the off-side rule (Section 7.1), recursive-descent and Pratt parsing (7.2), scoped symbol tables and bidirectional type checking (7.3), type inference over a lattice with a fixpoint (7.4), lowering to an explicit intermediate form (7.5), interval-based abstract interpretation for check elision (7.6), syntax-directed code generation (7.7), and translation validation by differential testing (7.10). It also touches the

part of compiler engineering that textbooks skip: interfacing generated code with a host runtime (pybind11), caching builds, and shipping the result as an installable product.

3. Problem statement, objectives, scope and limitations

3.1 Problem statement

Given a Python project, find the functions on which its run time is spent, translate each one that can be translated into C++17, prove on the program's own recorded calls that the translation behaves identically to CPython, install only the translations that pass and are actually faster, and report every place where the two languages' semantics could differ. Functions that cannot be translated are left running in Python, with a plain-English reason.

This is the Review 1 problem (a typed Python subset to readable native code, with a differential oracle and a divergence report) lifted from single files to whole projects, and from typed input to ordinary, untyped input.

3.2 Final approved scope

- **In scope:** the PySub language (a statically typed Python subset that has grown from the Review 1 core to include classes, dictionaries, sets, tuples, comprehensions, exceptions, f-strings and a part of the standard library); an in-house converter from untyped Python to PySub; the C++17 back end and its runtime; library mode (pybind11 extension modules); the Studio application (scan, trace, rank, fix, compile, verify, benchmark, splice, export); a Windows installer.
- **Out of scope, by design:** third-party C-extension libraries (NumPy, pandas, PyTorch, OpenCV), generators, `eval`/`exec`, threads, file and network I/O, regular expressions, unseeded randomness, non-ASCII text, and any back end other than C++17.

3.3 Objective-compliance table

Table 3.1 compares each measurable objective set in Reviews 1 and 2 with what was achieved. Where a target was missed or changed, the status says so.

Table 3.1. Objective compliance. Evidence for every value is in Appendix F.

#	Objective	Target (Review 1/2)	Achieved (Review 3)	Status
O1	Specify PySub formally	At least 40 EBNF productions; every construct implemented or excluded	`docs/grammar.ebnf` has 33 named rules (Review 1 reported 41; we could not reproduce that count). It covers the v0.1 core only; later constructs are specified by prose rules from the compiler source.	Partly met
O2	Reject ill-typed programs with positioned diagnostics	At least 95% of at least 60 negative programs rejected; no false rejections	88/88 rejected, each with the expected diagnostic text; 78/78 valid programs accepted (no false rejection).	Met
O3	Preserve observable behaviour against CPython	At least 98% of at least 120 programs identical	Oracle 78/78 (100%) on 78 programs, below the 120 target. Independent gates add 91 untyped programs (91/91 identical, no LLM) and 22 snippets (21 pass), and a generator adds sampled programs (Section 9.2). Rate met; the oracle count was not.	Rate met, count not
O4	Catalogue and report every divergence	At least 7 classes; every site positioned; at least 90% exact or guarded	14 classes (D1 to D14); 1280 positioned notes on the corpus, 42.6% exact, 56.7% guarded, 0.7% divergent.	Exceeded
O5	Competitive native code	Median at least 8x over CPython on at least 15 programs	Kernel benchmark: median 10.8x on 10 kernels, 9 of them 1.5x to 29.7x faster, and one string-keyed dictionary kernel about 29 times SLOWER (linear-scan dictionaries, Section 11.4). Oracle suite: median 15.2x on 78 single-run programs.	Met on the median; one kernel fails
O6	Guard elision measured (Review 2)	Implemented and a measured speedup	Measured with and without the pass on 10 kernels (Section 10.2): ratios 0.85x to 1.17x, scattered around 1.0. No consistent gain was demonstrated.	Implemented; benefit not demonstrated
O7	Classes, dict and try/except (Review 3 target)	Supported and tested	Classes with inheritance and dunders, dict, set, tuple, comprehensions, try/except/finally, assert, f-strings and a library subset, all in the oracle.	Met
O8	Region-based reclamation (Review 3 target)	Memory reclaimed without output change	Library mode frees every allocation of a call when it ends (library check: RSS grew 0 KB over 3000 calls). Standalone binaries still never free (D7).	Partly met

#	Objective	Target (Review 1/2)	Achieved (Review 3)	Status
O9	WebAssembly back end (Review 2 promise)	Second back end from one IR	Not delivered. Replaced by the Studio pipeline (Section 3.4).	Dropped
O10	Whole-project tool (Review 3, new)	Scan, rank, convert, verify, splice a real project, with no wrong output	See Table Table 10.3: zero wrong outputs among replay-verified functions; the end-to-end program output hash was compared before and after.	Met
O11	Installable product (Review 3, new)	A package someone else can install	`PYTHIA-Studio-Setup-1.1.0.exe` (88.8 MiB, per-user) and a 211.7 MiB portable zip, built by `packaging/windows/build.py`. We did not re-run the installer in this session.	Met (build not re-run)

3.4 Review 2 to Review 3: what changed and why

The faculty asked specifically for improvement since Review 2. Figure 3.1 shows the change in one picture and Table 3.2 lists it item by item.

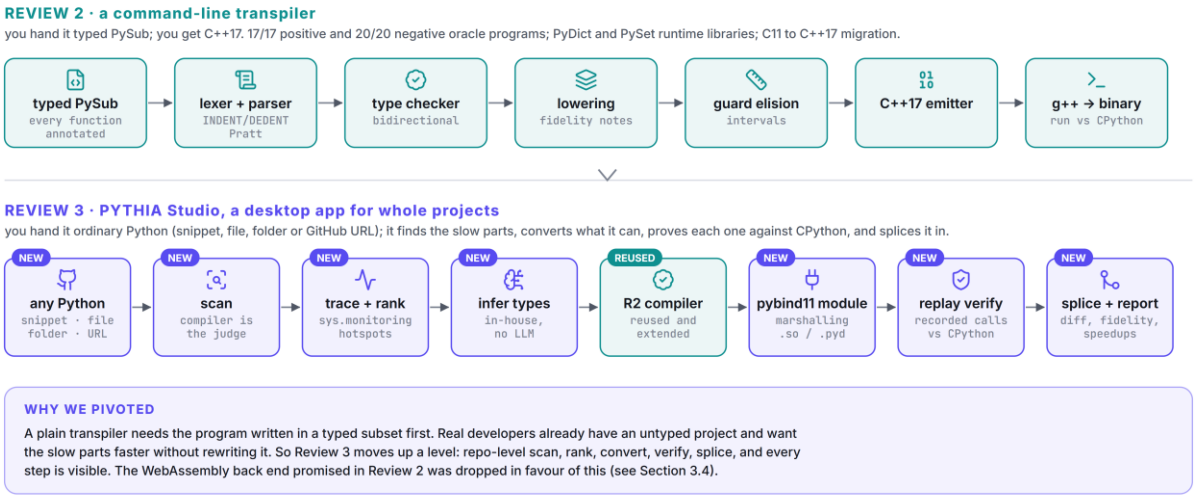


Figure 3.1. From a command-line transpiler (Review 2) to the Studio application (Review 3).

Table 3.2. Review 2 compared with Review 3.

Area	At Review 2	At Review 3	Why
Input	Typed PySub: every function annotated by hand	Ordinary Python: snippet, file, folder or GitHub URL; annotations are inferred	Developers have untyped projects, not PySub programs
Unit of translation	One whole program	One function at a time inside a larger project, spliced back	Speed up hot spots without rewriting the project
Types	Written by the author	In-house inference: observation, whole-program fixpoint, search (no LLM)	Deterministic, free, offline
Output	C++17 executable	pybind11 module (.so / .pyd) behind a one-line decorator, plus exportable C++	The project keeps running in Python
Correctness evidence	Differential oracle on 17 positive and 20 negative programs	Oracle (78 / 88) plus replay of the program's own recorded calls for every converted function	Evidence about the user's code, not only ours
Language	Scalars, lists, PyDict and PySet libraries	Plus classes, tuples, comprehensions, try/except, sets, f-strings and a standard-library subset (heapq, bisect, collections, itertools, functools)	Real code uses them
Guard elision	Implemented, barely measured	Measured on ten kernels with and without the pass (Section 10.2)	Review 2 gap
Memory	Never reclaimed (D7)	Per-call arena in library mode; standalone binaries unchanged	Review 1 plan for Review 3
Interface	Command line	Studio: pipeline stepper, diff, fidelity gutter, per-function journey, playground, report	Make every stage inspectable
Delivery	Clone and run	Windows installer, portable zip, wheel, Docker image	Usable by someone who did not write it
Back ends	C++17 (WebAssembly promised for Review 3)	C++17 only; **WebAssembly dropped**	Time went to the Studio pipeline; see Section 3.4

Why we pivoted. A transpiler that takes a hand-annotated subset is a good compiler exercise, but it is not how anyone would use the idea. A developer does not have a PySub program; they have a project. The question that matters to them is not "can this subset compile?" but "which of my functions can be sped up, how do I know the fast version is right, and what do I do about the rest?". Answering it needed three things the Review 2 tool did not have: a way to find the hot functions (tracing and ranking), a way to type untyped code (in-house inference), and a way to establish trust (replay on the program's own calls). The compiler core is unchanged in kind and reused; the new work sits around it.

Targets that changed. (1) The WebAssembly text back end promised for Review 3 was **dropped**. A second back end would have demonstrated retargetability but would not have helped a developer speed up a Python project, and the team's remaining time went to the Studio pipeline. (2) The oracle corpus target of 120 positive and 60 negative programs was met for the negative suite (88) and not for the positive oracle suite (78); the positive count is supplemented by an independent 91-program untyped gate and a grammar-directed generator (Section 9). (3) Region-based reclamation was delivered only for library mode (a per-call arena); standalone binaries still never free memory (divergence D7).

3.5 Limitations (summary)

The full list, with measured impact, is Section 11. In one sentence: PYTHIA converts pure-Python computation over ints, floats, bools, ASCII strings, lists, dicts, sets, tuples and simple classes; it will not touch code that depends on C-extension libraries, dynamic features or I/O, and its verification is differential testing on recorded inputs, not a proof.

4. Background and related work

4.1 Concepts

Off-side rule and layout tokens. Python's block structure is indentation, so the token stream is not context-free until the lexer turns indentation changes into explicit INDENT and DEDENT tokens. **Pratt parsing** (top-down operator precedence) handles expressions with a binding-power table instead of one grammar rule per precedence level. **Bidirectional type checking** (Pierce and Turner, local type inference) pushes an expected type inward where one is known and synthesises a type bottom-up where it is not. **Abstract interpretation with intervals** (Cousot and Cousot) computes a sound range for each integer expression and is used here to remove overflow checks that cannot fire. **Differential testing** (McKeeman) runs two implementations on the same input and compares behaviour; applied to a translator it is a form of **translation validation**: the translation is checked per program, not proved correct for all programs.

4.2 Prior tools and comparison

Nine existing systems address part of the problem. Figure 4.1 compares them with PYTHIA on the criteria that matter for this project; the sources for each cell are the tools' own documentation, listed in Section 14. We tried to say what each tool does better than PYTHIA, and the list is not short.

TOOL · OUTPUT	INPUT IT NEEDS	Readable output	Checked vs CPython	Reports divergence	Whole repo automatic	Unconvert-ible code still runs	No LLM or internet	Visual stage app
Cython C/C++ extension module	Python + C type declarations	○	○	○	○	●	●	○
Numba JIT machine code (in memory)	Python + @jit, NumPy style	–	○	○	○	○	●	○
PyPy alternative interpreter + JIT	unmodified Python	–	○	○	●	◐	●	○
mypyc C extension module	type-annotated Python	○	○	○	◐	●	●	○
Nuitka C program linked to libpython	any Python source	○	○	○	●	●	●	○
Codon native executable (LLVM)	Python-like, not drop-in	○	○	○	○	◐	●	○
Pythran C++ native module	Python subset + export comments	–	○	○	○	○	●	○
Shed Skin C++ program or module	implicitly typed Python 3 subset	–	○	○	○	○	●	○
LPython LLVM / C / C++ / WASM (alpha)	type-annotated Python subset	–	○	○	○	○	●	○
LLM translation C++ source text	any Python	●	○	○	○	○	○	○
PYTHIA readable C++17 + pybind11 module	untyped Python, supported subset	●	●	●	●	●	●	●

● yes ◐ partly / with caveats ○ no – not applicable or not assessed

Figure 4.1. Comparison with existing tools. Filled circle: yes; half: partly or with caveats; empty: no. Cells reflect each tool's official documentation (Section 14); "readable output" is the team's own assessment.

- **Numba** needs no annotations and is excellent on NumPy-style numeric code, but it is a JIT aimed at array-oriented, math-heavy code and needs a decorator per function; it gives no per-function check against CPython and no divergence report.
- **PyPy** runs unmodified programs with full Python semantics (including unbounded integers) at about 4x CPython on average according to its own site, with no setup at all. It is a different interpreter, so it needs no conversion and produces no code to read; it also does not give a C++ artefact.
- **Cython** and **mypyc** compile real Python (with optional static declarations or standard type hints) to C extension modules that keep Python semantics; they are more complete than PYTHIA and handle arbitrary imports. They do not verify or report divergence, and the user supplies the types.
- **Nuitka** compiles whole programs to C linked against libpython, claiming every Python construct; its speed-ups are modest, and the output depends on CPython.
- **Codon**, **Pythran**, **Shed Skin** and **LPython** compile a Python-like language or subset to native code (LLVM or C++) and can be faster than PYTHIA's output because they own their whole runtime. They require code that fits their subset or typing model: Codon's low-level documentation describes its standard `int` as a 64-bit signed integer, and it presents itself as a Python-like language rather than a drop-in replacement (we could not retrieve its "differences" page, so the second point is the team's reading); Shed Skin says existing code may need a lot of work; LPython is alpha.

- **LLM translation** (asking a chat model to rewrite Python as C++) handles any code and gives readable output, but nothing checks that the result behaves like the original, results differ between runs, and it needs an online service.

4.3 The identified gap and our novelty

Read across Figure 4.1: tools that handle untyped code (Numba, PyPy, Nuitka) do not give an auditable, readable artefact; tools that give readable native code (Codon, Pythran, Shed Skin, LPython) need code written for them; and none of the nine checks its output against CPython or tells the user where the semantics may differ. PYTHIA's contribution is the combination, and each part is stated here only as far as the evidence in Section 10 supports:

1. **Verified, not asserted.** Every converted function is run against CPython on calls recorded from the user's own program (topped up with generated inputs when too few were recorded, and labelled as such); results, mutated arguments and exception types must match, floats bitwise. A function that disagrees is rejected automatically. This is differential testing, not a proof (Section 11).
2. **The Semantic Fidelity Report.** A machine-generated, source-positioned list of every site where Python and C++ semantics could differ, each with its divergence class (D1 to D14) and mitigation. None of the surveyed tools documents a per-site report of this kind.
3. **Whole project, hands-off.** Point it at a folder or a GitHub URL; it scans every function, traces the program, ranks hot spots, converts what it can, explains in plain English why the rest was left alone, and splices compiled code back so the project keeps running.
4. **In-house type inference for untyped code, with no language model.** Deterministic, free, offline and reproducible: the same input always gives the same annotations. An optional LLM fallback exists, is off unless a key is configured, and cannot bypass the compiler or the replay check.
5. **Readable, exportable C++17 and an IDE-style application** that shows every stage, with a diff for every change.

What we do not claim: PYTHIA is not faster than Codon or Pythran on code those tools accept, it does not cover as much of Python as PyPy, Cython or Nuitka, and its "verified" label means "agreed with CPython on the calls we replayed".

5. Requirements and technology stack

5.1 Environment used for every measurement in this report

Table 5.1. Hardware and software environment.

Item	Value (detected on the machine that produced the results)
Machine	Intel(R) Core(TM) Ultra 9 185H (16 cores, 22 threads), 15.5 GiB RAM
Operating system	Microsoft Windows 11 Home Single Language 10.0.26300; WSL2 Ubuntu, kernel 6.6.87.1-microsoft-standard-WSL2
Python	Windows: 3.13.16 (the interpreter bundled in the installer); WSL: 3.14.4 (oracle, benchmarks)
C++ compiler	Windows: g++.exe (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r2) 16.2.0; WSL: c++ (Ubuntu 15.2.0-16ubuntu1) 15.2.0
Flags	`-std=c++17 -O2 -flto` (oracle and kernels); `-O3 -flto -fPIC -fvisibility=hidden` (extension modules)
Libraries	pybind11 3.1.0; FastAPI 0.115.12, Uvicorn 0.34.3, Pydantic 2.11.5; Node 22.14.0, React 18.3.1, Vite 5.4.21, TypeScript 5.9.3, Monaco 0.52.2
Capture tools	Playwright 1.61.0 driving Microsoft Edge; Matplotlib 3.10.8; python-docx 1.2.0; Microsoft Word for the PDF export
Repository	commit `95bbb1b` on `main`; measurements on 2026-10-24

5.2 Software and libraries

Table 5.2. Technology stack.

Layer	Choice (version used)	Role
Compiler	Python 3.13.16, standard library only	Lexer, parser, type checker, lowering, optimiser, emitter, converter

Layer	Choice (version used)	Role
Target and runtime	C++17; `pyrt.hpp/.cpp`, `pydict.hpp`, `pyset.hpp`	Python-semantics runtime: int64 checks, floor division, strings, lists, dicts, sets, exceptions
Bridge	pybind11 3.1.0	Extension modules and marshalling (`pyrt_bridge.hpp`)
Toolchain	g++ (g++.exe (MinGW-W64 x86_64-ucrt-posix-seh on Windows)	Builds the generated source; bundled on Windows
Server	FastAPI 0.115.12, Uvicorn 0.34.3, WebSockets	REST, event stream, static UI
UI	React 18.3.1, TypeScript 5.9.3, Vite 5.4.21, Tailwind, Monaco 0.52.2, Zustand	Studio application; icons from Lucide
Packaging	Inno Setup 6.7.3, hatchling wheel, Docker	Installer, portable zip, wheel, container image
Test and measurement	oracle, `check_basics.py`, `check_snippets.py`, `property_test.py`, `fuzz.py`, `stress.py`	Section 9

5.3 Test programs and corpora

The project needs no external data set. Its inputs are self-authored: 78 positive and 88 negative PySub programs (`tests/positive`, `tests/negative`), 91 untyped everyday programs (`scripts/basics`), 22 LeetCode- and GfG-style snippets (`scripts/snippets`), two bundled demo projects (`demo/textlab`, `demo/quanta`), a grammar-directed program generator (`tools/fuzz.py`), a property-based differential harness (`scripts/property_test.py`), and a stress harness that downloads public GitHub repositories (`scripts/stress.py`). Section 9 describes each.

5.4 Why this, not that

Table 5.3 records each major technology choice, the alternative we considered, and the reason. Where the repository records the reason (in `studio/DECISIONS.md` or a commit message) we cite it; where the reason is our own engineering judgement we say so.

Table 5.3. Technology choices and their alternatives.

Decision	Chosen	Considered	Why
Compiler language	Python 3	C++, Rust	The compiler manipulates trees and strings, not hot loops, and Studio must import and run the user's modules in process (tracing, replay). Our judgement.
Lexer and parser	Hand-written (`lexer.py`, `parser.py`)	ANTLR, Lark, PLY	Full control of the off-side rule and of positioned error recovery, and no third-party import in the compiler core. Cost: each new construct is a hand-written hook.
Target	C++17 (migrated from C11 on 4 Sept 2026, pull request 1)	C11, LLVM IR	pybind11 needs C++; templates give monomorphic containers; lambdas give single evaluation; the output stays readable source and g++ does the optimisation. C11 had needed GNU statement expressions at three sites (Review 1, Section 5.4). LLVM IR would not be readable and would need LLVM at run time.
Python bridge	pybind11 3.1.0	ctypes, cffi, raw C-API, nanobind	Automatic conversions and exception translation with one header-only dependency built by the same g++. ctypes and cffi need a C ABI and hand-written list and string marshalling; the raw C-API is long. We did not evaluate nanobind.
Correctness evidence	Differential testing (oracle and replay)	Unit tests, formal proof	The specification of a translator is "behaves like CPython", and the cheapest executable form of that is to run both. A proof of the whole pipeline was out of reach; we say so (Section 11).
Type source	In-house inference	LLM translation	We tried it first: a Groq/Anthropic fixer and an LLM Playground converter (commits `dfe6942`, `cf2cfb8`). The free tier is about 200,000 tokens per day per model (`STATUS.md`), answers vary between runs, it needs a network, and every answer still has to pass the compiler and the replay. The LLM path remains as an optional fallback (commit `e4a4804`).
Server	FastAPI with WebSockets	Flask, Django	First-class async WebSockets for the event stream, Pydantic models, and one small ASGI app that serves the UI bundle too.
Desktop UI	React, Vite, TypeScript, Monaco in an Edge/Chrome `--app` window	Tkinter, Qt, Electron	Monaco gives a real diff editor; the same UI runs in any browser and as a PWA; no second runtime to ship, so the installer is 88.8 MiB with Python and a compiler inside. We did not benchmark Electron.

Decision	Chosen	Considered	Why
Windows toolchain	Bundled MinGW-w64 GCC 16.2 (WinLibs, UCRT)	MSVC	The build flags (`-O3 -flto -fvisibility=hidden`) and the runtime are developed against GCC, and a redistributable zip can be shipped inside the installer; MSVC Build Tools are a separate installation. Our judgement.
Installer	Inno Setup 6.7.3	WiX / MSI, NSIS	One scripted `.iss`, per-user install without administrator rights, free.
Tracing	`sys.monitoring` (3.12+), `sys.setprofile` fallback	`sys.settrace`	Per-function call events at lower cost than line tracing; `settrace` is not used anywhere in the repository (`_tracer_boot.py:66`, `observe.py`).
Accept rule	Measured replay speedup of at least 1.1x	Static heuristics only	A function that compiles is not necessarily faster; the number decides (`DECISIONS.md`, B2).

6. System design

6.1 Architecture and data flow

PYTHIA has two tiers. The **compiler** (`pythia/`) turns PySub source into C++17. **Studio** (`pythia/studio/`) wraps it in a pipeline that works on whole projects and a web user interface that shows each stage. Figure 6.1 (next page) draws the complete system in six layers; the numbered arrows are walked through below.

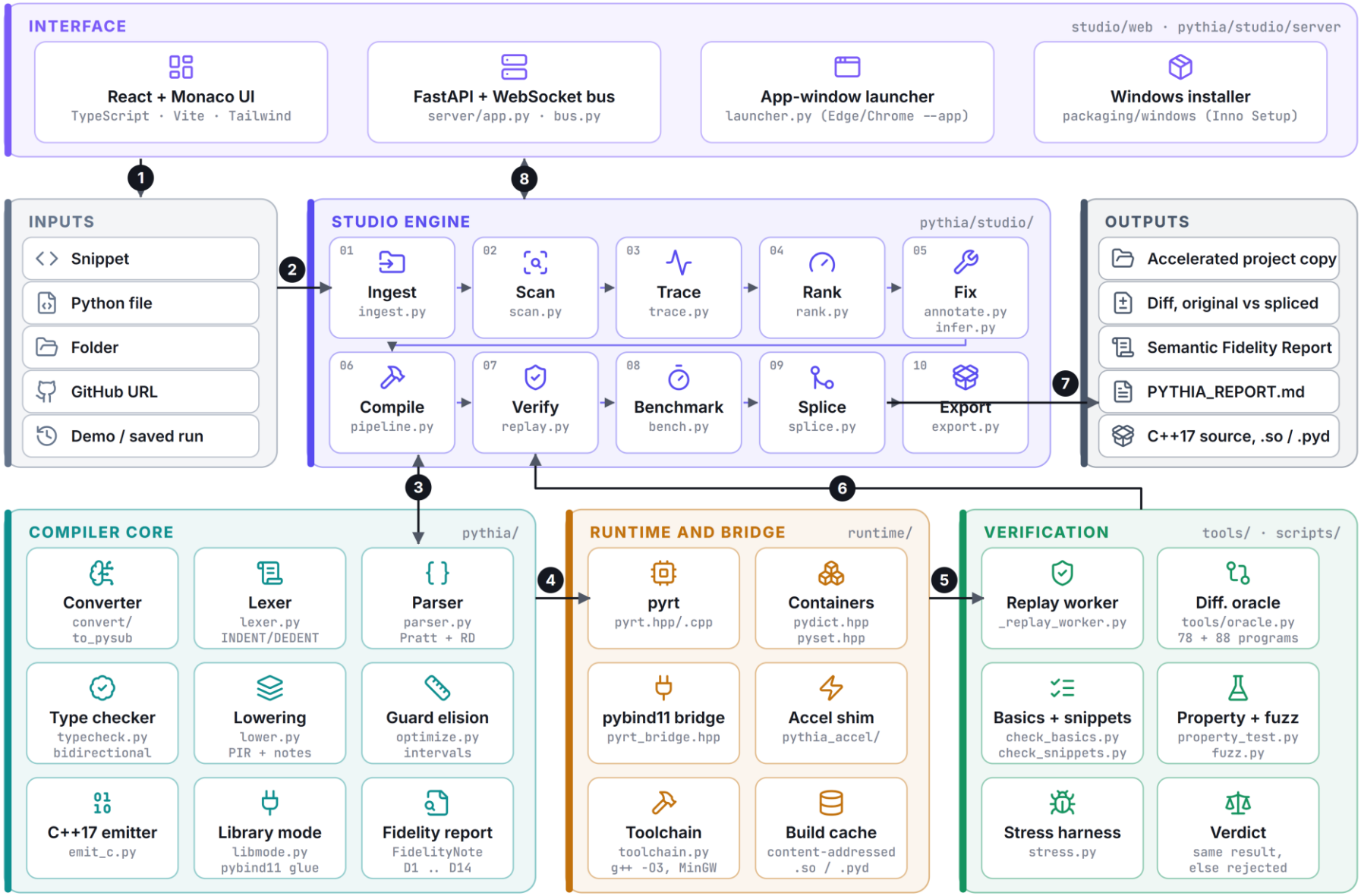


Figure 6.1. PYTHIA architecture. Arrows 1 to 8 are described in the text; stage numbers 01 to 10 in the Studio engine are the pipeline order.

1. **Input.** The user gives the UI a snippet, a file, a folder, a GitHub URL, or opens the bundled demo or a saved run. The server (FastAPI) accepts it as a job.
2. **Ingest.** The project is copied into an isolated workspace with a pristine `original/` and a `work/` copy that receives the splice; GitHub URLs are fetched as a zip archive, so git is not required. Long Windows paths are skipped and recorded.
3. **Compiler as judge and as builder.** *Scan*, *Fix* and *Compile* call the compiler core: the real PySub parser and type checker decide whether a function is eligible (`scan.py`), the converter adds missing annotations (`convert/`, `studio/infer.py`), and the lexer, parser, type checker, lowering, guard elision and emitter produce C++17 and pybind11 glue (`libmode.py`).
4. **Build.** The C++ translation unit is compiled with the bundled or system g++ against the runtime (`pyrt.hpp/.cpp`, containers, `pyrt_bridge.hpp`) into a `.so` or `.pyd`. Builds are content-addressed so identical source is never rebuilt.
5. **Replay.** A worker process imports the original function and the compiled module and feeds both the recorded calls.
6. **Verdict.** Results come back to the Verify and Benchmark stages: agreement and a speedup of at least 1.1x keep the function, anything else is rejected or declined with the numbers.
7. **Outputs.** Accepted functions get a one-line decorator in the `work/` copy; the diff, the Semantic Fidelity Report, `PYTHIA_REPORT.md`, the C++ source and the compiled modules are exported. The original folder is never modified.
8. **Interface.** Every stage emits typed events on a WebSocket bus (contiguous sequence numbers, persisted to `events.jsonl`), so the UI can play a run live, replay a saved one, or step through it in presenter mode.

6.2 Modules and interfaces

Table 6.1. Main modules. Line counts are non-blank lines measured with `docs/review3/src/collect_facts.py`.

Module	Files (lines of code)	Responsibility	Interface
Front end	`lexer.py`, `parser.py`, `parse_ext.py`, `parse_iter.py`, `parse_types.py`, `ast_nodes.py` (2,325)	Tokens with layout, AST, positioned syntax diagnostics with recovery	`parse(src, path)` returns the module and errors
Type system	`typecheck.py`, `types_ext.py`, `classes_ext.py`, `builtins_ext.py`, `tuples.py`, `comprehensions.py` (6,136)	Scopes, bidirectional checking, classes, tuples, comprehensions, built-ins	`typecheck(mod, path)` returns the checker and errors
Lowering and optimiser	`lower.py`, `optimize.py` (787)	PIR with explicit coercions, fidelity notes, interval guard elision	`lower(mod, tc)`, `elide_guards(mod)`
Back end	`emit_c.py`, `emit_ext.py`, `lib_ext.py`, `libmode.py`, `classes_lib.py`, `toolchain.py`, `cli.py`, `driver.py`, `stages.py` (3,006)	C++17 emission, pybind11 glue, builds, command line	`emit(...)`, `build_library(src, funcs, out)`
Converter	`pythia/convert/` (2,687)	Untyped Python to PySub: observation, fixpoint inference, search	`to_pysub(source, stdin)`
Runtime	`runtime/` (2,853)	Python semantics in C++, containers, marshalling, accel shim	headers included by generated code
Studio back end	`pythia/studio/` including server (7,307)	Ingest, scan, trace, rank, fix, compile, verify, benchmark, splice, export; REST and WebSocket	REST, WebSocket, events
Studio UI	`studio/web/src` (9,531, TypeScript)	Stepper, editors, diff, journey, playground, report	REST and WebSocket client
Tools and scripts	`tools/`, `scripts/` (3,754)	Oracle, generator, checks, stress, property tests	command line
Packaging	`packaging/windows/` (448)	Installer assembly, launcher, Inno Setup script	`build.py`

6.3 Key data structures

- **Token** (`lexer.py:32`): kind, value, line, column. Layout is explicit: INDENT, DEDENT, NEWLINE, EOF.

- **AST and typed AST** (`ast_nodes.py`): one dataclass per construct; every node carries `line`, `col` and a `ty` slot that the type checker fills, so the typed AST is the same tree.
- **Ty** (`typecheck.py:23`): `name` plus optional `elem`, `key`, `elts` (list, dict, tuple, optional, class, set). The widening order is bool < int < float (`widens_to`, line 54; `join`, line 71).
- **FidelityNote** (`lower.py:26`): line, column, divergence class, detail, mitigation, severity (exact, guarded or divergent).
- **Interval** (`optimize.py`): `None` for unknown or a `(lo, hi)` pair of Python ints, kept inside plus or minus 2^62.
- **FunctionRecord** (`studio/models.py`): one record per function, holding its eligibility, status (`pending`, `verified`, `guarded`, `fixable`, `not_eligible`, `declined`, `failed`), reason, trace, score, every source form (original, annotated, PySub, C++, wrapper), build, replay, benchmark and splice results, and the 16-step *journey* the UI shows.
- **Event** (`studio/server/bus.py`): typed, sequenced messages (`job.start`, `stage`, `step`, `fix.done`, `error`, ...). The reducer in the UI is a pure function over this stream, which is why live, replayed and mocked runs render identically.

6.4 Design decisions

The decisions below are recorded in `studio/DECISIONS.md` and the commit history; Table 5.3 covers technology choices.

- **The compiler is the judge.** `scan.py` does not list PySub's features; it extracts each function into a standalone module and asks the real parser and type checker, then translates diagnostics into plain English. New compiler features therefore need no scanner change.
- **Splice by decorator, never by rewrite.** `@_pythia_accelerated` is added above the original `def`; the body stays, the diff is small and reversible, and if the compiled module is missing or raises `FallbackError` the Python body runs.
- **Never splice a slowdown.** A function whose measured speedup is below 1.1x is `declined` and kept as a measured probe, so the decision is backed by a number.
- **Cold code is skipped.** Functions below 0.2% of traced run time are neither built nor sent to any fixer; without this a 270-function project would build about 80 modules.
- **Copy semantics at the boundary.** Arguments are copied in and mutated lists and dicts are written back; anything not representable raises `FallbackError`; integers outside int64 raise `OverflowError`.
- **Correctness rule.** *CPython-exact or rejected*: an accepted construct reproduces byte-identical stdout and the same exception type and message, otherwise the compiler rejects it with a message that names the reason.

7. Algorithms and methodology

For each stage this section states what it does, the algorithm, its complexity, and the trade-off we accepted. Source locations are from commit `95bbb1b`. *n* is the number of tokens or AST nodes of the unit being processed.

Figure 7.1 follows one real function from a third-party repository through the stages this section describes; the numbers are from the run in Section 10.4.

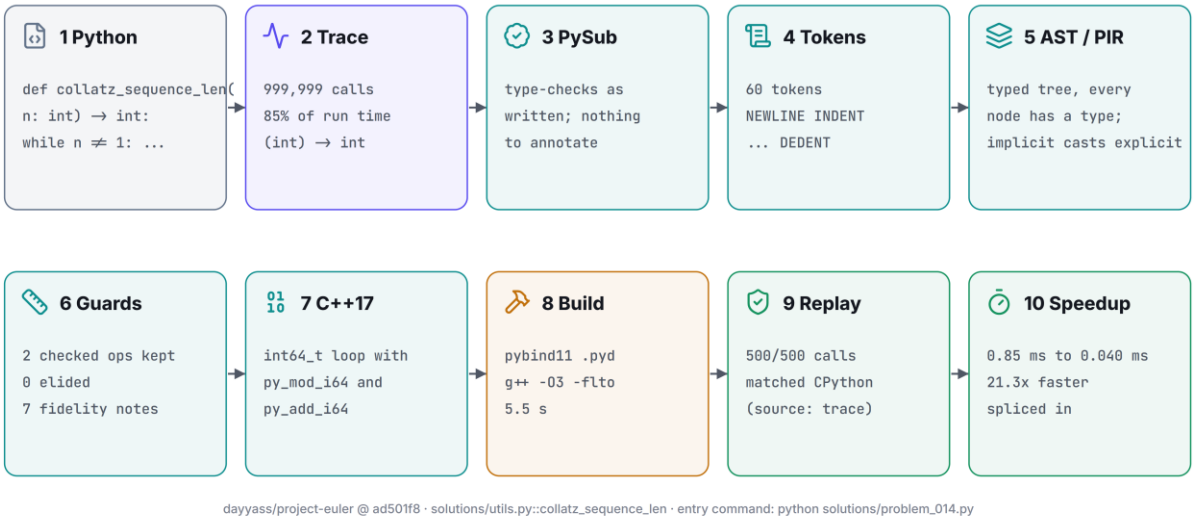


Figure 7.1. One function's journey end to end: dayyass/project-euler, collatz_sequence_len (Python, trace, PySub, tokens, typed tree, guards, C++17, build, replay, speedup).

7.1 Lexing and the off-side rule (`lexer.py`)

The scanner keeps a stack of indentation widths, initially `[0]`. At the start of a logical line it measures the indentation: a larger value pushes the stack and emits INDENT; a smaller value pops and emits one DEDENT per popped level, and the new width must equal a width still on the stack. Blank lines and comment-only lines carry no layout, brackets suppress layout entirely (`paren_depth`), `\` joins lines, and `;` acts as a statement separator. At end of file the lexer emits a final NEWLINE and one DEDENT per open block. Operators are matched longest-first (`**=` before `**` before `*`). **Cost:** one pass, O(n) time, O(depth) space. **Trade-off:** tabs are not interpreted by the lexer; the converter expands them to four spaces on entry to avoid mixed-indentation surprises (commit `125bd67`).

7.2 Parsing: recursive descent plus Pratt (`parser.py`)

Statements use one recursive procedure per construct. Expressions use a Pratt loop (`parse_expr`, line 512) over a binding-power table: `or` 1, `and` 2, comparisons 4, `|` 5, `^` 6, `&` 7, shifts 8, `+ -` 10, `* / // %` 12, `**` 16 (right associative, so `a ** b ** c` groups to the right). Chained comparisons are desugared as `a < b and b < c`; if the middle operand is not pure, the parser reports it rather than evaluate it twice. The conditional expression binds loosest. Syntax errors use **panic-mode recovery**: `synchronize()` skips to the next NEWLINE, DEDENT or statement keyword and always consumes at least one token, so a parse can report several errors and can never loop. **Cost:** O(n), one token of lookahead beyond the current token for `not in` and `is not`. **Trade-off:** hand-written, so each new construct (f-strings, comprehensions, slices) is a hook added by hand; the benefit is precise, friendly diagnostics and no dependency.

7.3 Bidirectional type checking (`typecheck.py`)

`synth` computes a type bottom-up; `check_expr(e, expected)` pushes an expected type inward. The distinction matters for exactly the cases that bottom-up alone cannot type: an empty `[]` or `{}` takes its element type from the annotation or parameter it flows into; `[1, 2.5]` is checked against `list[float]` and widened. Function signatures are collected before bodies are checked, so mutual recursion works. Scopes are a parent-linked chain (`Scope`, line 82). Implicit conversion follows the lattice bool, int, float, plus subclass-to-base and optional rules that the extension modules register. Every error carries line and column and, where it helps, both types in conflict. **Cost:** O(n) node visits, each doing an O(depth) scope lookup. **Trade-off:** each variable has one static type for its lifetime, which is what makes the output unboxed and fast and also what rejects Python code that rebinds a name to another type.

7.4 In-house type inference for untyped Python (`pythia/convert/`)

Most real Python has no annotations, so PYTHIA must supply them. It does so without a language model, in four steps, and edits only the original text (comments and strings never change).

1. **Observation** (`observe.py`). Run the program once on its stdin under `sys.setprofile` and record the runtime types of every function's arguments and locals and of module globals. This fills holes that static reasoning leaves open, and the program's own output is kept to compare against the converted program later.
2. **Whole-program fixpoint** (`infer.py`, `Engine`, line 163). Every variable, return value, attribute and record field is a *slot*. Evidence from assignments, call sites, returns, method uses and iteration is *joined* into slots on the lattice of `types.py` (scalars, `list`, `set`, `dict`, `tuple`, `opt`, class instances, records) until no slot changes. Conflicting evidence becomes an absorbing `bad` element, and an int-on-some-paths, float-on-others slot becomes `num`, which is refused wherever it would change printed output.
3. **Search** (`search.py`). Parameters that no call fixes (LeetCode-style `class Solution` methods) are tried against up to 400 candidate type assignments ordered by syntactic hints (a `.append` call suggests a list, `.lower()` a string); the first assignment that makes the *whole program* type-check wins.
4. **The compiler decides.** The annotated program is parsed and type-checked by PySub's own checker; only then does it go on. Anything the checker rejects is reported with a plain-English reason, not guessed at.

Cost: each pass is O(n); the lattice has finite height (nested types are cut off at depth 5), so the iteration terminates. **Trade-off:** soundness over coverage. A function that returns `-1` on one path and a float on another is refused, because converting it would print `-1.0` where Python prints `-1`; the snippet `coin_change` in our own suite is rejected for exactly this reason (Section 9.4). Inside the Studio pipeline the same ladder runs per function: annotate from types observed by the tracer, then deterministic inference, and only then (if a key is configured) an optional LLM rewrite that must still pass the compiler and the replay.

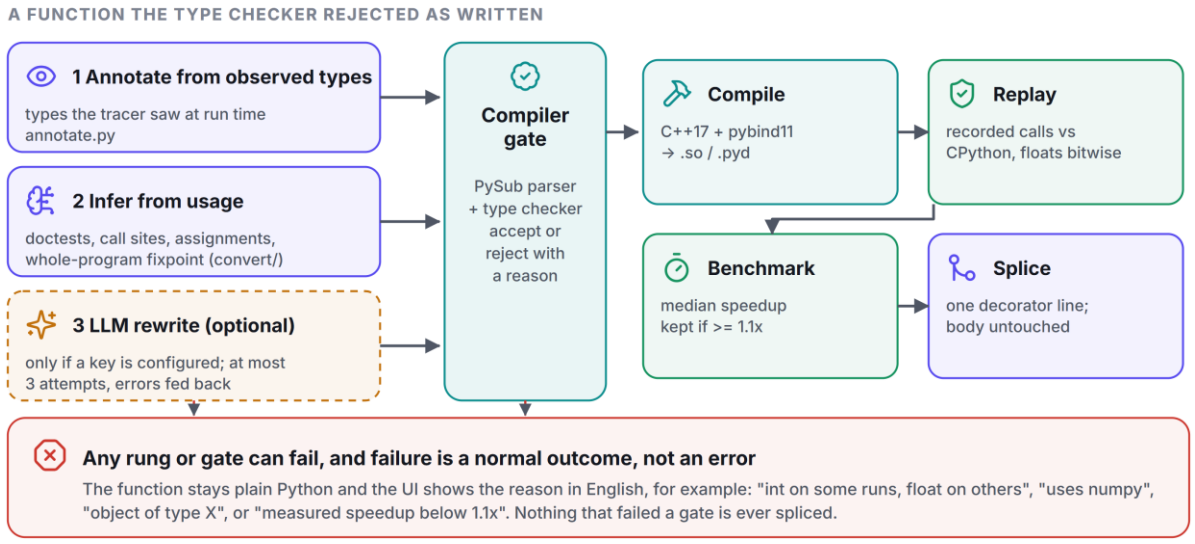


Figure 7.2. The fix-and-verify ladder applied to every function. Every rung is deterministic except the optional last one, and no rung can bypass the compiler or the replay.

7.5 Lowering and the divergence catalogue (`lower.py`)

`Lowerer` walks the typed AST and rewrites it into PIR, which uses the same node classes but obeys two invariants: every implicit conversion is an explicit `Coerce` node, and every site where C++ could behave differently from Python carries a `FidelityNote`. The catalogue has fourteen classes; Table 7.1 lists them.

Table 7.1. The divergence catalogue (D1 to D14) as implemented.

Class	Divergence	Treatment
D1	Unbounded `int` against `int64_t`	Guarded: checked `+ - *` raise `OverflowError`; guards removed where interval analysis proves them unreachable; ints beyond int64 at the boundary raise `OverflowError`.
D2	`//` floors toward minus infinity, C++ truncates	Exact: `py_floordiv_i64` and the float form.

Class	Divergence	Treatment
D3	`%` takes the sign of the divisor	Exact: `py_mod_i64`, including signed zero for floats.
D4	`/` is always true division	Exact: operands widened to double, `ZeroDivisionError` preserved.
D5	Negative indexing and bounds checks	Exact: index normalised and checked in the runtime; `IndexError` reproduced.
D6	`int ** negative int` changes the result type	Divergent: cannot be typed statically; reported at compile time, refused at run time.
D7	Object lifetime and reclamation	Guarded: standalone binaries never free; library mode frees everything allocated in a call when the call ends.
D8	Exception propagation across calls	Exact: C++ exceptions of one type (`PythonException`) caught in `main`. The note text in `lower.py` still describes the Review 1 `setjmp`/`longjmp` mechanism and is stale.
D9	Class fields zero-initialised before `__init__`	Guarded: an unset field reads as zero instead of raising `AttributeError`.
D10	`sum()` of an empty `list[float]`	Guarded: CPython returns int 0, compiled code 0.0; non-empty sums use the same compensated summation.
D11	Ordering of NaN in sort, min, max	Guarded: merge sort and timsort may place NaN differently.
D12	Interactive `input()`	Exact: prompt to stdout, one line from stdin, `EOFError` at end.
D13	Characters are ASCII	Guarded: non-ASCII input raises `FallbackError` at the boundary; `ord`/`chr` above 127 raise `PythiaSemanticError`.
D14	libm last-bit differences; user-defined ordering	Guarded: math functions call the toolchain's libm; sorting objects by `__lt__` calls it in a different order.

On the 78 positive oracle programs the compiler emitted 1280 positioned notes: 545 exact, 726 guarded and 9 divergent (Section 10.3). **Cost:** O(n). **Trade-off:** notes are emitted for every site, which is noisy; the UI folds the global notes and shows the rest as a gutter next to the diff.

7.6 Guard elision by interval analysis (`optimize.py`)

Python integers do not overflow; int64 does. PYTHIA keeps Python's meaning by compiling each `+`, `-`, `*` as a checked operation (`py_add_i64`, `py_sub_i64`, `py_mul_i64` in `pyrt.cpp`: explicit range tests) that throws `OverflowError`, and then removes the check where it provably cannot fire. The pass is abstract interpretation over intervals (Cousot and Cousot), run as a single forward pass. Literals give point intervals, `range(a, b)` bounds the induction variable, `len()` bounds indices, and results are kept only if they stay within plus or minus 2^62, which leaves two bits of headroom. A variable that is assigned in a loop and feeds back into itself (an accumulator) is widened to *unknown* before the body is analysed, which is what makes one pass sound without a fixpoint solver. For non-exposed functions, literal call-site arguments also bound parameters; exposed functions are called by Python with arbitrary ints, so that bound is dropped. **Soundness contract:** it may keep a guard that was not needed, never remove one that was. **Cost:** O(n). The measured effect is in Section 10.2; in short, it helps where indices and counters dominate and does nothing for accumulators.

7.7 C++17 emission (`emit_c.py`)

Emission is one syntax-directed walk of PIR. Parameters and locals are prefixed (`p_`), functions `f_`, temporaries `_tN`; primitive lists map to monomorphic runtime types (`PyListI64`, `PyListF64`, `PyListBool`, `PyListStr`), dictionaries with scalar key and value types to eight specialised dict types and otherwise to the generic `PyDictT`, and class instances to `std::shared_ptr`. Python's loop-variable rules are kept by driving `for ... in range` from a hidden counter and assigning the visible variable from it, so reassigning `i` inside the body cannot change the iteration. Expressions that must evaluate an operand once (`and`/`or` returning an operand, `min`, `max`) use immediately invoked lambdas. Listing 7.1 shows real output. **Cost:** O(n) for the walk; build time is dominated by g++.

```
#include "pyrt.hpp"

/* forward declarations (PySub allows mutual recursion) */
static int64_t f_total(PyListI64 *p_xs, int64_t p_n);

static int64_t f_total(PyListI64 *p_xs, int64_t p_n) {
    int64_t p_s = 0;
    int64_t p_i = 0;
```

```
    p_s = INT64_C(0);
    {
        int64_t _t1 = p_n;
        int64_t _t2 = 1;
        if (_t2 == 0) py_raise("ValueError", "range() arg 3 must not be zero");
        for (int64_t _t3 = INT64_C(0);
             _t2 > 0 ? _t3 < _t1 : _t3 > _t1;
             _t3 += _t2) {
            p_i = _t3;
            p_s = py_add_i64(p_s, py_mul_i64(PyListI64_get(p_xs, p_i), INT64_C(2)));
        }
    }
    return p_s;
    return 0;   /* unreachable unless the result is Optional */
}

static int pythia_main(void) {
    pyrt_set_py_minor(13);   /* match this CPython's error texts */
    try {
        py_print_begin();
        py_print_sep();
        py_print_i64(f_total([&]() { PyListI64 *_t5 = PyListI64_new(); PyListI64_append(_t5, INT64_C(1));
PyListI64_append(_t5, INT64_C(2)); PyListI64_append(_t5, INT64_C(3)); return _t5; }(), INT64_C(3)));
        py_print_end();
        return 0;
    } catch (const PythonException &exc) {
        py_print_exception(exc);
        return 1;
```

Listing 7.1. Output of `python -m pythia.cli e.py --emit-only` for a three-line kernel. The accumulator `p_s` keeps its guarded `py_add_i64`/`py_mul_i64` because it is loop-carried; the loop counter needs none.

7.8 Library mode and marshalling (`libmode.py`, `pyrt_bridge.hpp`)

Library mode compiles a module without `main()` and generates a pybind11 wrapper for the requested functions. Marshalling has *copy semantics*: arguments are converted to runtime objects, results converted back, and mutated lists and dicts are written back into the caller's object. What cannot be represented faithfully (a non-ASCII string, a float where an int is expected, a tuple for a list) raises `FallbackError` and the shim runs the original Python; an integer beyond int64 raises `OverflowError`. Each call runs inside a **per-call arena**: every runtime object allocated during the call is freed when it ends, which is the region-based reclamation planned in Review 1, delivered for library mode only. Builds are content-addressed on source and toolchain; the header `pyrt_bridge.hpp` is precompiled, which the repository records as cutting a cold build to about 9 s and a repeat build to about 4 s (cache hit about 20 ms). **Trade-off:** copying makes calls with large arguments expensive, which the ranker accounts for.

7.9 Tracing and hotspot ranking (`_tracer_boot.py`, `rank.py`)

The entry command (for example `python main.py`) runs with a tracer injected as `sitecustomize`. On Python 3.12 and later it uses `sys.monitoring`, which has much lower overhead than `sys.settrace`; earlier versions fall back to `sys.setprofile`. For each target function it records calls, cumulative and self time, observed argument and return types, element counts, and up to a bounded number of sampled (arguments, result) pairs. The ranker then scores `trace_share x eligibility x expected_benefit x (1 - boundary_penalty)`, where the penalty is `marshal / (marshal + saving)`, `marshal = calls x (1.5 us + 4 ns x elements)` and `saving = traced seconds x (1 - 1/benefit)`. A function whose predicted net saving is negative is declined with the numbers shown, and functions below 0.2% of run time are marked cold. The two constants are calibration knobs, not measurements of every machine.

7.10 Differential oracle and replay verification (`tools/oracle.py`, `replay.py`)

Two instruments check two different claims. The **oracle** checks the compiler: for every program in `tests/positive` it runs CPython and the compiled binary on the same stdin and demands byte-identical standard output and the same uncaught-exception class; for `tests/negative` it demands that the compiler reject the program with a diagnostic containing the text in the file's `# expect:` marker. The **replay** checks one conversion of one user function: the recorded calls (at least 50, topped up with synthesised inputs and then labelled as such) are run through the original and the compiled function in a worker process, and the return value, the exception class and the state of mutated arguments must match, with floats compared by bit pattern (`struct.pack('<d')`, so

NaN equals NaN). A function that disagrees is rejected; one that agrees is benchmarked (median over repeats) and kept only if it is at least 1.1x faster. **Cost:** linear in the number of calls. **Limit:** agreement on the replayed calls is evidence, not proof.

8. Implementation details

8.1 Code organisation

```
pythia/                     the compiler (Python)
  lexer.py  parser.py  ast_nodes.py    off-side rule, Pratt/recursive descent, AST
  typecheck.py  types_ext.py           bidirectional type checker and extensions
  classes_ext.py  tuples.py  comprehensions.py  builtins_ext.py
  lower.py  optimize.py                PIR, fidelity notes, interval guard elision
  emit_c.py (emit_cpp.py alias)  emit_ext.py  lib_ext.py   C++17 emitter
  libmode.py  classes_lib.py           pybind11 extension modules
  toolchain.py  paths.py  cli.py  driver.py  stages.py
  convert/                           in-house untyped-Python converter (no LLM)
  studio/                            Studio back end: pipeline, scan, rank, replay, splice, server/
runtime/                    C++ runtime: pyrt.hpp/.cpp, pydict.hpp, pyset.hpp, pyrt_bridge.hpp, pythia_accel/
                            (pyrt.c/.h: Review 1 C11 runtime, not used by the C++17 pipeline)
studio/web/                 React + TypeScript UI (dist/ committed)    studio/DECISIONS.md  CONTRACTS.md
packaging/windows/          installer build (build.py, launcher.c, .iss)
tests/positive  tests/negative   oracle corpora (78 and 88 programs)
tools/                      oracle.py  fuzz.py  fuzz_run.py
scripts/                    check_basics, check_snippets, property_test, stress, check_libmode, ...
demo/                       quanta (3,800 lines), textlab (480 lines), offline cache, sample run
docs/                       grammar.ebnf, Review 1 report/deck, review3/ (this report, viva guide, sources)
.github/workflows/          ci.yml (oracle, library mode, package), release.yml
```

Listing 8.1. Repository layout (selected). Counts are non-blank lines.

8.2 Module-wise notes

- **Front end.** The lexer and parser were written for Review 1; since then the parser grew hooks for slices, bitwise operators, f-strings, comprehensions, lambdas, tuples, classes and `try`, each in a small extension module (`parse_ext.py`, `parse_iter.py`, `parse_types.py`) so the core grammar function stays readable.
- **Type system.** `typecheck.py` is the core; `types_ext.py` (optional, isinstance, sets, defaults and keyword arguments, module globals), `classes_ext.py` (classes, inheritance, dunders, properties, dataclasses), `tuples.py`, `comprehensions.py` and `builtins_ext.py` register through a few dispatch chains. The shared chains are the places where parallel work conflicted most, and are recorded as such in `STATUS.md`.
- **Back end.** `emit_c.py` keeps its Review 1 file name (the C++17 migration reused it); `emit_cpp.py` is an eight-line public alias. The runtime is `pyrt.hpp/.cpp` plus `pydict.hpp`, `pyset.hpp` and `pyrt_bridge.hpp`; the C11 runtime (`pyrt.c/.h`) remains in the tree as the Review 1 artefact and is not used by the C++17 pipeline.
- **Studio back end.** `pipeline.py` runs the ten stages and emits events; `scan.py` (about 1,200 lines) extracts each function with its helpers and constants into a standalone module; `fixer.py` and `llm.py` hold the optional LLM path; `replay.py`, `_replay_worker.py` and `codec.py` implement verification; `splice.py` and `export.py` produce the accelerated copy.
- **Studio front end.** React 18 with TypeScript, Vite, Tailwind and Monaco, bundled with fonts and no CDN at run time; the compiled `dist/` is committed so the app runs without Node. A mock engine (`?mock=1`) serves the whole API in the browser for UI development.

8.3 Build process

Compiler pipeline. `python -m pythia.cli prog.py --run` parses, type-checks, lowers, elides guards, emits C++17 and calls g++ (`-std=c++17 -O2 -flto` in the oracle; `-O3 -fPIC -fvisibility=hidden -shared` for extension modules) together with `runtime/pyrt.cpp`. `--fidelity` prints the divergence report; `--lib f,g` builds a pybind11 module.

Windows installer. `packaging/windows/build.py` assembles a relocatable CPython (NuGet package, 3.13), a pruned WinLibs MinGW-w64 GCC 16.2 toolchain (UCRT), the PYTHIA wheel contents, the UI bundle and the prebuilt demo modules, then calls Inno Setup 6.7.3 to build `PYTHIA-Studio-Setup-1.1.0.exe` (88.8 MiB, per-user, no administrator rights) and a portable zip (211.7 MiB). Extension modules link statically against `python3X.dll`, so a

`.pyd` needs nothing else. GCC's default-manifest link spec breaks on install paths with spaces, so the bundled toolchain renames that object and PYTHIA passes it explicitly.

8.4 Integration

Three GitHub Actions jobs run on every push (`.github/workflows/ci.yml`): the differential oracle, the library-mode self-check against CPython, and a packaging check that builds the wheel, installs it into a fresh virtual environment, boots the server and runs the offline demo job. A second workflow builds the wheel, sdist and container image on a version tag. We did not run CI in this session; the oracle, basics, snippet and library checks were run locally (Section 9).

9. Testing strategy and test cases

Testing is layered so that each layer catches a different kind of error. Table 9.1 lists the suites; Table 9.2 gives representative cases with expected and actual results taken from runs in this session.

Table 9.1. Test suites.

Suite	What it checks	Size	Result (this session)
Oracle, positive	Compiled binary against CPython: stdout and exception class	78 programs	78/78 identical
Oracle, negative	Compiler must reject, with the `# expect:` text	88 programs	88/88 rejected
Untyped basics (`check_basics.py`)	In-house conversion, compile, compare; no LLM	91 programs	91/91 identical
Snippets (`check_snippets.py`)	LeetCode and GfG style, untyped	22 snippets	21/22 (`coin_change` refused)
Library mode (`check_libmode.py`)	pybind11 module against CPython: values, exceptions, write-back, classes, leak	74 + 22 calls	OK
Grammar-directed generator (`fuzz.py`)	Sampled well-typed programs, oracle-style comparison	150	150/150 identical, 0 mismatches
Property tests (`property_test.py`)	Ten algorithm categories, randomised inputs	10 categories	50/50 identical
Kernel benchmark	Ten fixed kernels, median of runs, outputs compared	10 kernels	outputs identical
Studio runs	Whole projects through the real pipeline with replay	5 runs	see Section 10.4
Stress (`stress.py`)	Public repositories file by file, no LLM	not run	cut: not re-run this session

9.1 Positive and negative corpora

Positive programs exercise each language feature and each divergence class; the oracle compares CPython and the compiled binary byte for byte, including the exception class for programs that are meant to fail (`t12_err_index`, `t13_err_zerodiv`, `t77` to `t82`). Negative programs each carry an `# expect:` marker and are rejected only if the diagnostic contains the expected text, so the suite tests message quality, not just rejection.

9.2 Untyped basics, snippets, property tests, generated programs

`scripts/check_basics.py` converts every untyped program in `scripts/basics` with the in-house converter and **no LLM** (the script blanks the API key variables), compiles it, and compares output and exception type with CPython on the same stdin. `scripts/check_snippets.py` does the same for LeetCode and GfG-style snippets. `scripts/property_test.py` runs ten algorithm categories on randomised inputs, and `tools/fuzz.py` samples well-typed programs from the grammar in the spirit of Csmith.

9.3 Representative test cases

Table 9.2. Representative cases. `Actual` columns come from the logged runs in Appendix F.

ID	Category	Input / description	Expected	Actual	Status
TC-01	Positive	Operator precedence: print(2 + 3 * 4 ** 2)	CPython: 50	PYTHIA: 50	PASS
TC-02	Boundary	Floor division and modulo with negative operands	CPython: -4 1 -4 -1	PYTHIA: -4 1 -4 -1	PASS

ID	Category	Input / description	Expected	Actual	Status
TC-03	Boundary	Float repr: 0.1 + 0.2 and 1e16	CPython: 0.30000000000000004 1e+16 1000000000000000.0	PYTHIA: 0.30000000000000004 1e+16 1000000000000000.0	PASS
TC-04	Negative	Type mismatch in assignment: x: int = 'hello'	CPython: hello	PYTHIA: rejected at 1:10: 'x' is annotated int but the initialiser has type str	PASS
TC-05	Boundary	Index out of range: xs[10] on a 3-element list	CPython: [IndexError]	PYTHIA: [IndexError]	PASS
TC-06	Boundary	Division by zero: 1 // z	CPython: [ZeroDivisionError]	PYTHIA: [ZeroDivisionError]	PASS
TC-07	Boundary	range with step zero	CPython: [ValueError]	PYTHIA: [ValueError]	PASS
TC-08	Regression	Unbounded recursion	CPython: [RecursionError]	PYTHIA: [RecursionError]	PASS
TC-09	OOP	Class, method call, field update	CPython: 12	PYTHIA: 12	PASS
TC-10	Limit (D1)	Integer beyond int64: print(2 ** 62 * 2)	CPython: 9223372036854775808	PYTHIA: [OverflowError]	LIMIT
TC-11	Limit (D13)	Non-ASCII string length: len('héllo')	CPython: 5	PYTHIA: 6	LIMIT
TC-12	In-place	Library mode: swap list ends inside compiled code, caller's list	CPython: [4, 2, 3, 1]	PYTHIA: [4, 2, 3, 1]	PASS
TC-13	Limit (D13)	Library mode: word_len('héllo') with non-ASCII input	CPython: 5	PYTHIA: FallbackError (shim then runs the Python body)	PASS
TC-14	Limit (D1)	Library mode: list element 2**70 (beyond int64)	CPython: works (bignum)	PYTHIA: OverflowError	LIMIT

9.4 Stress and regression

`scripts/stress.py OWNER/REPO` downloads a public repository as a zip, pushes every file through converter, type checker, C++ build and run against CPython, and reports build failures and output differences (which must be zero) and the ranked reasons for rejection. Section 10.5 gives this session's results.

A regression we found and kept honest. During this session `scripts/check_snippets.py` reported 21 of 22 snippets, not the 22 of 22 recorded in `STATUS.md`. Bisecting with `git worktree` showed the change arrived in commit `210a599` ("Expand in-house conversion for lambdas and closures"). `coin_change` initialises `dp` with `float('inf')` and returns `-1` when unreachable. Before the commit the converter typed the function as returning `float`, which compiles but prints `3.0 -1.0` where CPython prints `3 -1` (we re-ran the old conversion to confirm); after it, the converter reports conflicting types and refuses. The refusal is the CPython-exact behaviour, so we left it and corrected the number rather than the code.

10. Results and evaluation

10.1 Correctness

Table 10.1. Correctness results, all produced in this session (Appendix F gives commands and dates).

Gate	Result	What it demonstrates
Differential oracle, positive	78/78 (100%)	Compiled output and exception class equal CPython's on every program.
Differential oracle, negative	88/88 (100%)	Every invalid program is rejected with the intended message.
Untyped basics, zero LLM	91/91 (100%)	The in-house converter plus compiler reproduces CPython on everyday programs without annotations.
Snippets, zero LLM	21/22	One snippet is refused because converting it would change printed output.
Library mode	OK	74 calls and 22 class calls compared with CPython; no RSS growth over 3000 calls.
Generated programs	150/150 identical, 0 mismatches	Programs sampled from the grammar, compared with CPython.
Property tests	50/50 identical	Randomised inputs in ten algorithm categories.

Every gate reports agreement with CPython or an explicit refusal; none reports a silent difference. The one number that fell below its recorded value, 21 of 22 snippets, is a refusal: the refused program would have printed `3.0 -`

`1.0` where CPython prints `3 -1`. The oracle's own speed column is one run of a whole process per program and mixes very short programs with long ones, so it is reported (median 15.2x) but the kernel benchmark below is used for performance claims.

10.2 Compiler performance on a fixed set of kernels

To avoid cherry-picking, ten kernels were fixed before measuring (`docs/review3/src/kernels/`): integer loops, a sieve, dense matrix multiply, recursion, dynamic programming on strings and arrays, floating-point iteration, sorting, and a string-keyed dictionary count. Each was compiled with the oracle's flags, run 5 times under CPython and as a native binary, and the **median whole-process wall time** is reported (CPython start-up is 104.6 ms of every Python figure; the native binaries start in about a millisecond). Outputs were byte-identical in every run. Table 10.2 gives the numbers and Figure 10.1 plots them.

Table 10.2. Kernel benchmark. Medians of the runs shown above; min and max are in `evidence/bench_kernels.json`.

Kernel	CPython (ms)	PYTHIA (ms)	Speedup	Without guard elision (ms)	Elision gain	Guards elided / kept
collatz	1822	66	27.4x	66	0.99x	0 / 2
sieve	381	42	9.1x	41	0.97x	2 / 1
matmul	237	20	11.6x	21	1.01x	1 / 9
nqueens	106	11	9.6x	9	0.85x	2 / 13
levenshtein	360	247	1.5x	238	0.96x	0 / 13
mandelbrot	507	17	29.6x	20	1.17x	0 / 0
fib	258	26	9.9x	28	1.09x	0 / 3
bubble	365	12	29.7x	14	1.15x	0 / 7
knapsack	295	20	14.5x	18	0.89x	1 / 7
wordcount	485	14260	0.03x	14242	1.00x	0 / 4

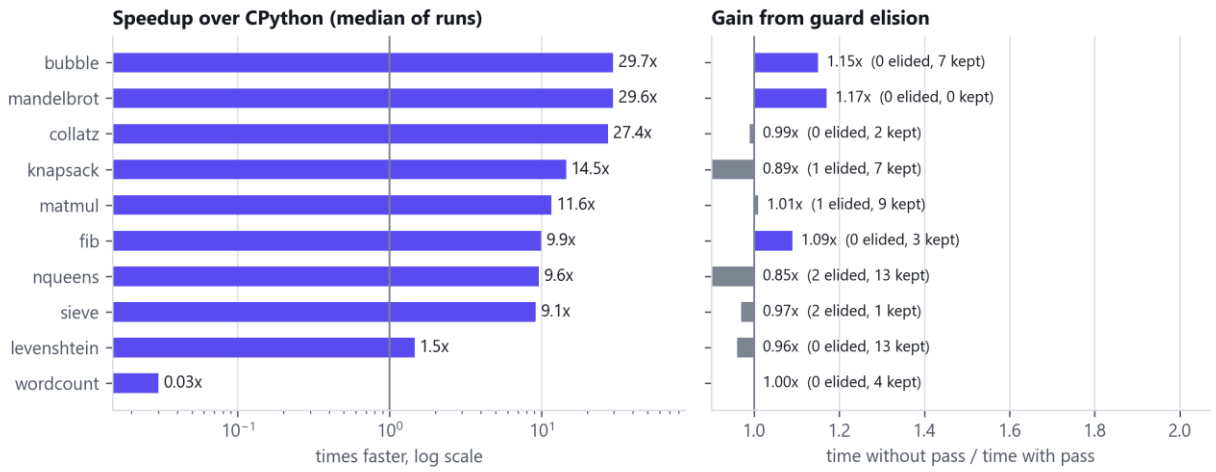


Figure 10.1. Speedup of the PYTHIA binary over CPython per kernel (left, log scale) and the effect of interval-based guard elision (right).

Nine of the ten kernels are 1.5x to 29.7x faster. Indexed integer and float loops (collatz, mandelbrot, bubble sort) gain most; recursion, the sieve, matrix multiply and N-queens gain about 9 to 12x; Levenshtein gains only 1.5x because the string-indexing path allocates. The geometric mean over all ten is 6.59x and the median 10.8x; every output was byte-identical to CPython's. **One kernel is a failure and we report it:** the string-keyed dictionary count (`k10_wordcount`, about 2,000 distinct keys, 1.5 million updates) ran in 14260 ms compiled against 485 ms in CPython, about 29 times slower. The cause is in the runtime: the eight specialised dictionary types look keys up by scanning a vector (`DICT_IMPL`, `runtime/pyrt.cpp:131`), so each lookup is O(number of keys), where CPython's is O(1). Dictionaries are correct but not fast; a hash index would fix it and is the first item in Section 13.2. **Guard elision.** The right-hand panel of Figure 10.1 compares each kernel with and without the interval pass. The ratio lies between 0.85x and 1.17x, scattered on both sides of 1.0, which is the size of run-to-run noise at five runs. **We therefore cannot claim a measured benefit from the pass.** It removes very few checks on these kernels (Table

Table 10.2), and with `-O2 -flto` the compiler can often inline the checked helpers and predict their branches. The Review 1 report anticipated a factor of two; we did not observe it.

10.3 Semantic Fidelity Report statistics

Across the 78 positive programs the compiler emitted 1280 positioned notes, a median of 11 per program. 42.6% are *exact* (the runtime reproduces CPython's rule), 56.7% *guarded* (identical unless a documented condition occurs, for example an integer beyond int64) and 0.7% *divergent*; the divergent ones are all D6 (`int ** negative int` changes the result type), which the compiler reports and refuses at run time. Figure 10.2 shows the distribution by class. D1 (integer width) and D5 (negative indexing and bounds) dominate because every integer operation and every subscript is a site.

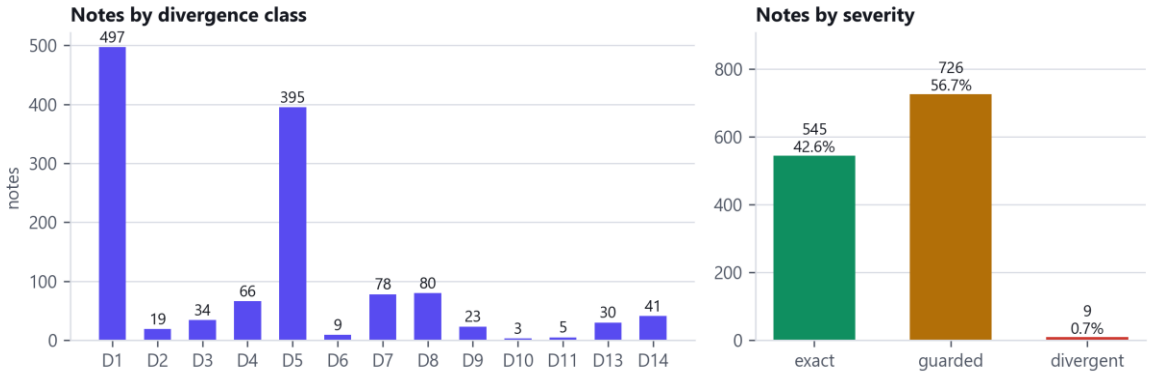


Figure 10.2. Fidelity notes over the positive corpus, by divergence class (left) and severity (right).

10.4 Studio on whole projects

Each project was run through the real application path (the HTTP API that the UI uses) on a clean server with no LLM key, with its trace command and the build cache cold unless marked warm. A function is counted *accelerated* only if its replay matched on every call and its measured speedup was at least 1.1x. **textlab**: 35 functions found, 9 accelerated, 2 declined, 22 not eligible; whole job 232 s; program 17.2 s to 6.5 s (2.64x), output hash identical. **project-euler (problem 14)**: 102 functions found, 1 accelerated, 21 declined, 64 not eligible; whole job 51 s; program 8.9 s to 0.8 s (11.58x), output hash identical. **project-euler (problem 10)**: 102 functions found, 1 accelerated, 21 declined, 64 not eligible; whole job 33 s; program 7.1 s to 1.3 s (5.43x), output hash identical. **project-euler (problem 30)**: 102 functions found, 1 accelerated, 20 declined, 64 not eligible; whole job 78 s; program 9.9 s to 8.8 s (1.13x), output hash identical. **textlab (before print fix)**: 35 functions found, 9 accelerated, 2 declined, 22 not eligible; whole job 238 s; program 17.4 s to 6.8 s (2.56x), output hash DIFFERENT.

Table 10.3. Studio runs, zero-LLM path, build cache cold unless noted.

Project	Funcs found	Verified / guarded	Declined	Not eligible	Geomean fn speedup	End-to-end
textlab	35	9 (2 / 7)	2	22	9.7x	2.64x
project-euler (problem 14)	102	1 (0 / 1)	21	64	21.3x	11.58x
project-euler (problem 10)	102	1 (0 / 1)	21	64	1.8x	5.43x
project-euler (problem 30)	102	1 (0 / 1)	20	64	1.3x	1.13x
textlab (before print fix)	35	9 (2 / 7)	2	22	9.6x	2.56x (output differs!)

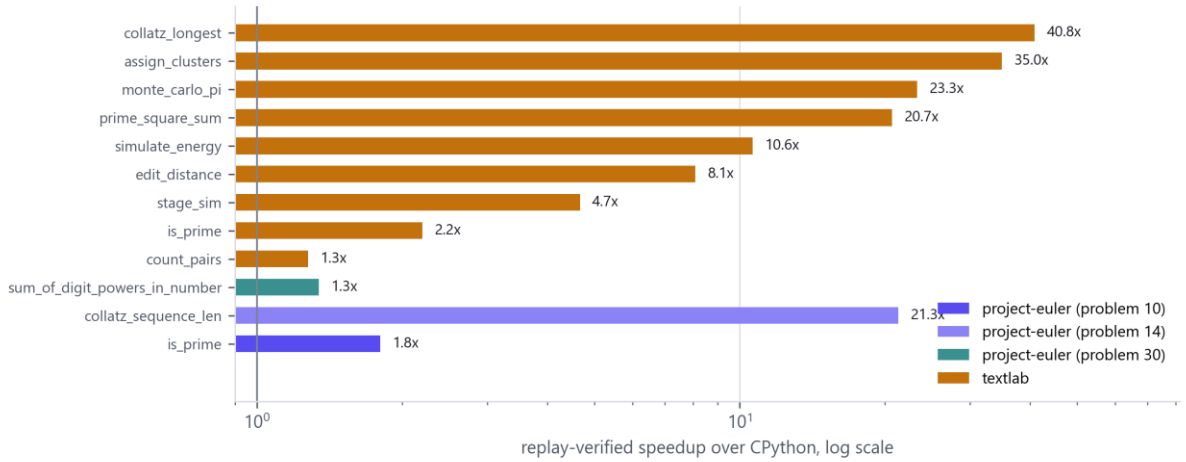


Figure 10.3. Per-function speedups from the Studio runs (replay-verified, log scale).

10.5 Stress test on public repositories

The stress harness (`scripts/stress.py`) was **not re-run in this session**. The figures in `STATUS.md` (for example 17 of 17 files on one repository and 58 to 67% on three others, with zero converter crashes) were measured earlier on repositories the team had tuned on and are not reproduced here, so we make no stress-test claim. The independent evidence from this session is the demo-repository screening in `LIVE_DEMO.md`: about twenty small public repositories were cloned, five went through the full pipeline, and every function that was spliced matched CPython.

10.6 Interpretation and threats to validity

- **Where the speedup comes from.** Unboxed 64-bit integers and doubles, monomorphic contiguous lists, and no interpreter dispatch. Loop kernels reach 30x; the linear-scan dictionary is a slowdown (Section 10.2).
- **Amdahl applies.** Per-function speedups are far larger than whole-program ones because the rest of the program keeps running in Python (Table Table 10.3).
- **The boundary can erase the gain.** Copying arguments in and out costs time proportional to their size; the ranker declines functions where that dominates, and the replay's measured number is the final judge.
- **Single machine, single toolchain family.** All timings are from one laptop (Intel(R) Core(TM) Ultra 9 185H) with GCC 15 under WSL2 (kernels) and GCC 16 on Windows (Studio). We did not test other CPUs, other operating systems, or clang.
- **Whole-process timing.** Kernel times include process start-up (CPython about 104.6 ms). That favours the compiled binary slightly on short kernels; all kernels run for at least 0.1 s under CPython.
- **Recorded-call verification.** Agreement is on calls the program made plus labelled generated inputs; unexercised paths are untested. The oracle and generator widen the evidence on the compiler itself, not on a user's function.
- **Corpus authorship.** Most test programs were written by the team (and by AI assistants working for the team). They may share our blind spots; the stress harness on third-party repositories is the independent check.

11. Error handling, limitations and known issues

11.1 Error handling

Compile-time problems are reported as `line:col: Kind: message` with the source line and a caret, from the lexer, parser and type checker; one run reports several syntax errors thanks to panic-mode recovery. At run time the emitted program wraps `main` in a handler that prints Python's exception name and message (`IndexError: list index out of range`, `ZeroDivisionError`, `OverflowError`, `RecursionError`) so that the oracle can compare exception classes; message text follows the minor version of the CPython that runs the comparison, because 3.11 to 3.14 word some errors differently. In Studio, every stage failure is a typed event that fails one

function (never the job), and an exception inside compiled code reaches Python as the matching Python exception or as `FallbackError`, which the shim catches to run the original body.

11.2 Limitations with measured impact

Table 11.1. Limitations, their observed effect and the way forward.

Limitation	What happens	Observed impact	Remedy (future work)
C-extension libraries (NumPy, pandas, PyTorch, OpenCV)	Functions that use them are `not eligible`; they stay in Python.	Their own loops are already native; the speedup is lost only on the Python around them. Not measured on a NumPy project.	Call CPython from compiled code; a NumPy subset on Eigen or xtensor; LibTorch for PyTorch.
Dynamic typing and metaprogramming	`eval`, `exec`, `getattr`, a variable that changes type, int-or-float results are rejected with a reason.	Snippet `coin_change` refused: 21 of 22 snippets pass. Converting it would print `3.0 -1.0` instead of `3 -1`.	Compile one version per observed type signature, or `std::variant` unions.
64-bit integers (D1)	Checked arithmetic raises `OverflowError`; an int beyond int64 at the boundary raises too.	`2 ** 62 * 2`: CPython prints 9223372036854775808, PYTHIA `[OverflowError]`. Library mode: list element 2**70 gives `OverflowError`.	Promote to a bignum when a check fires.
ASCII-only strings (D13)	Non-ASCII input to a compiled function raises `FallbackError`; the original Python runs.	`len('héllo')`: CPython 5, standalone `6`; library mode `FallbackError (shim then runs the Python body)`.	UTF-8 strings with code-point indexing.
Memory reclamation (D7)	Standalone binaries never free memory. Library mode frees everything allocated in a call when it ends.	RSS stayed at 28640 KB after 200000 calls that each built a 500-element list and string (before: 28640 KB).	Region allocator or reference counting for standalone code.
Dictionary speed	Specialised dicts use a linear scan per lookup.	String-keyed word count, about 2,000 keys: about 29 times slower than CPython (Table Table 10.2).	Hash index in `DICT_IMPL`.
Boundary cost	Arguments are copied in and mutated containers copied back.	A call to a trivial compiled function costs about 0.15 us against 0.038 us for `len`; a 100,000-element list costs about 1.18 ms per call. The ranker declines functions where this dominates (Table Table 10.3).	Zero-copy buffers, data kept on the C++ side, fusing neighbouring functions.
Verification is differential testing	Agreement on recorded and labelled generated calls only.	A path the traced run never took is untested; fuzzing found 0 mismatches in 150 programs, which bounds nothing about other programs.	Fuzz around recorded inputs, validate each pass, prove one pass formally.
Unsupported constructs	Generators, nested functions, decorators, `with`, `global`, `*args`, `async`, `match` (Section 11.3).	Reported per function with a reason; 0 crashes of the converter on the stress repositories recorded in `STATUS.md`.	Closures by lambda lifting; generators by state machines.
Randomness, files, OS, regex, threads	Refused with a reason (`explain.py`).	Programs that print random numbers cannot be compared with CPython unless seeded.	Reproduce CPython's Mersenne Twister for seeded use.
Compile-time latency	g++ runs once per module and is cached by content.	Library check: 15.05 s cold, 5.63 s for a new module with the runtime built, 27 ms on a cache hit. A cold Studio run took 51 s on the demo repository and 232 s on TextLab.	Ship prebuilt modules for known projects; build in the background.
Needs a C++ toolchain	Bundled on Windows (GCC 16.2); on Linux and macOS the user installs g++.	Windows installer 88.8 MiB.	Ship a compiler in the Linux and macOS packages too.
Python to C++ only	One front end, one back end.	WebAssembly back end dropped.	The IR is language-agnostic after the front end.

11.3 Unsupported constructs today

The Studio scanner rejects, with a plain-English reason, any function that is `async` or uses `await`; is decorated; takes `*args` or `**kwargs`; defines a nested function; is a generator (`yield`); uses `global` or `nonlocal`, `with`, `match`, an import inside the function, the `@` operator, bytes or complex literals, or an integer literal beyond 64 bits; calls dynamic built-ins (`type`, `getattr`, `setattr`, `eval`, `exec`, `open`, `input` and similar); or reaches a module it cannot compile (third-party packages, `os`, `sys` I/O, `json`, `random`, `re`, threads). The Playground's converter is more permissive about lambdas and some closures, but verification applies there too. This list is taken from `scan.analyze()` (line 940) and `explain.py`.

11.4 Known issues

- **Snippet `coin_change`** is refused (int and float on different paths); 21 of 22 snippets pass, not the 22 of 22 recorded in `STATUS.md` (Section 9.4).
- **`docs/grammar.ebnf` is the v0.1 core** (33 named rules). The implemented language has grown past it, and the extensions are documented by prose rules generated from the compiler source (`pysub_rules.py`), not by an updated grammar. Review 1 counted 41 productions; a count of named rules gives 33.
- **Stale text in the fidelity catalogue:** the D8 note still describes `setjmp`/`longjmp`, and the D7 note says region-based reclamation is "the Review 3 target" although library mode now has it.
- **`STATUS.md` lists three stress-harness bugs** (a C++ build failure on a dict-of-tuples case, and two int-versus-float printing cases that replay correctly rejects). They were recorded before this session and were not re-investigated here.
- **Windows long paths:** repositories with paths over 260 characters have those files skipped on ingest.
- **A stale server is the usual cause of impossible-looking errors.** The launcher refuses to reuse a server with a different build fingerprint, but a forgotten WSL server on the same port can still answer first.
- **Oracle timings are single runs** of whole processes, so very short programs mostly measure process start-up; the kernel benchmark in Section 10.2 is the number to quote.

12. Member-wise contribution

How to read this section. The repository's first commit (`84f1251`, 4 September 2026) already holds the Review 2 code base as a single commit by the repository owner, so git cannot attribute the Review 1 and 2 modules to individuals. Their ownership below follows the Review 1 responsibility matrix and the team's contribution log (`CONTRIBUTIONS.md`). What git *can* show is recorded as evidence: `git shortlog` gives 138 commits by the repository owner (135 under one address, 3 under a GitHub no-reply address), 3 by `Copilot` (the dictionary pull request), 1 by `AaryanGupta01` and 1 by `Jahnavi`. Three pull requests from teammates' forks are the only non-owner work in history. The Studio work (17 to 19 October 2026) was committed by the owner, using AI coding assistants (Section 14); its ownership split below is **proposed** and must be agreed by the team.

Table 12.1. Member-wise contribution.

Member	Technical ownership (Reviews 1 and 2)	Evidence	Integration and testing
Vishal Vivek 24BCE2377 lead	Front end (grammar, lexer with the off-side rule, parser, AST, diagnostics), command line, repository, integration. Review 3: the Studio engine, server, web UI and installer.	138 of 143 commits (`git shortlog`); every Studio commit (`2f46833` to `95bbb1b`, 17 to 19 Oct); merges of the three pull requests (`086b6f8`, `079bcb5`, `4ef0102`) and of eight workstream branches (for example `c8ca929`, `176f5d0`). Front-end files are in the first commit `84f1251` with `Owner: Member 1` headers.	Integrated all workstreams, kept the oracle green, wrote `STATUS.md` and `DECISIONS.md`; built and ran the installer.
Rujuta Mangesh Kulkarni 24BDS0459	Symbol table, bidirectional type checker and local inference (`typecheck.py`), negative corpus and diagnostic quality, dictionary support.	Pull request 2 from `rujuta-student`, merged as `079bcb5` (9 Sept): `runtime/pydict.hpp` (351 lines), `tests/dict_test.cpp`, five documents. The commits in that pull request are authored by `Copilot` (Co-authored-by Copilot App); the git record shows an AI assistant wrote them on her branch. `typecheck.py` and `tests/negative` are in the first commit.	Negative suite (88 programs, each tested for message text); type-checker soundness fixes found by the oracle.
Koliparthy Venkata Jahnavi 24BDS0132	Divergence catalogue and Semantic Fidelity Report (`lower.py`), `pyrt` runtime, interval-analysis guard elision (`optimize.py`), PySet.	Pull request 3 from `Jahnavikoliparthy`, merged as `4ef0102` (9 Sept): commit `ddc975b` by Jahnavi adds `runtime/pyset.hpp` (533 lines) and `tests/set_test.cpp` (366 lines). `lower.py`, `optimize.py` (header: Owner Member 3, Review 2 deliverable) and `pyrt.c/.h` are in the first commit.	Fidelity statistics and the guard-elision measurement in Section 10.

Member	Technical ownership (Reviews 1 and 2)	Evidence	Integration and testing
Aaryan Gupta 24BDS0134	C++ emitter, C11 to C++17 migration, differential oracle, benchmark harness, program generator, CI, corpus.	Pull request 1 from `AaryanGupta01`, merged as `086b6f8` (4 Sept): commit `9264cbb` "Migrate backend from C11 to C++17" (6 files, +270/-48: `emit_c.py`, `emit_cpp.py`, `pyrt.cpp`, `pyrt.hpp`, `oracle.py`, `cli.py`). `emit_c.py`, `tools/oracle.py`, `tools/fuzz.py`, `tools/fuzz_run.py` (Owner: Member 4) and the first CI workflow are in the first commit.	Oracle (78/88), generator (150/150 in this session), CI (three jobs), kernel benchmark design.

Table 12.2. PROPOSED ownership split for the Review 3 Studio work, aligned with each member's compiler phase. Not evidenced by git; the team must confirm it.

Member	Proposed ownership of the Studio work (to be agreed)
Vishal Vivek	Studio shell: `pipeline.py` (the ten stages), `server/app.py` and `bus.py` (REST and WebSocket events), the React UI, `launcher.py` and the Windows installer.
Rujuta Mangesh Kulkarni	In-house type inference and eligibility: `convert/infer.py`, `types.py`, `search.py`, `observe.py`; `scan.py` and `reasons.py` (the compiler as judge, plain-English reasons).
Koliparthy Venkata Jahnavi	Fidelity and the boundary: the fidelity gutter and bottom panel, `libmode.py` and `pyrt_bridge.hpp` (marshalling, `FallbackError`, `OverflowError`, arena), `rank.py` (boundary-cost model).
Aaryan Gupta	Verification and measurement: `replay.py`, `_replay_worker.py`, `codec.py`, `bench.py`, `stress.py`, `property_test.py`, the oracle and CI.

13. Conclusion and future work

PYTHIA set out to answer one question: can a student-built compiler turn the slow parts of an ordinary Python project into native code *and* show why the result can be trusted? The work delivered a complete compiler pipeline (Reviews 1 and 2), and in Review 3 a system around it that scans, traces, ranks, types, converts, compiles, verifies, benchmarks and splices a real project, with a user interface that exposes every step and a Windows installer. Against the Review 1 and 2 objectives (Table Table 3.1), the type-checking, equivalence-rate, divergence-catalogue and performance objectives are met or exceeded; the grammar specification lags the language, the oracle corpus is 78 programs against a target of 120, reclamation covers library mode only, and the WebAssembly back end was dropped. On a third-party repository the system found the hot function, compiled it (21.3x), declined its caller on a measured 0.81x, and sped the program up 11.6x with identical output; on other candidates it found nothing worth compiling and said why, which is the intended behaviour. The honest summary of the result is that verified conversion of a well-defined subset works, is useful on compute-heavy pure-Python code, and is bounded by the language subset, the 64-bit and ASCII choices, and the quality of the recorded calls.

13.1 What we learned

- A compiler's *front end* is the easy half of a transpiler. The hard half is the contract with the host language: what happens at the boundary, on overflow, on a type that changes, and how to *show* that the output is right.
- Reporting beats hiding. Making every divergence a positioned note turned an unanswerable question ("is it faithful?") into a list the team could shrink, and it caught a regression in this very session (Section 9.4).
- A measurement harness is part of the product. The ranker's constants, the 1.1x threshold and the cold-code cut-off exist because a function that compiles is not the same as a function worth compiling.
- Parallel work on shared dispatch chains needs one integrator and a green oracle; the hook-order merge conflicts recorded in `STATUS.md` are the cost of skipping either.

13.2 Future work

- **Bignum promotion** on overflow instead of `OverflowError`, and UTF-8-aware strings with code-point indexing, to remove the two largest semantic gaps (D1, D13).
- **Boundary cost.** Zero-copy buffers (the buffer protocol for lists of ints and floats), keeping data on the C++ side across calls, and fusing neighbouring hot functions into one compiled unit.

- **Specialisation.** Compile one version per observed type signature, or `std::variant` unions, so that functions that see both int and float are converted instead of refused.
- **Reclamation for standalone binaries** (regions or reference counting), so D7 can be closed outside library mode.
- **Verification depth.** Property-based fuzzing of each converted function around the recorded inputs, translation validation per pass, and a machine-checked proof of one pass in the style of CompCert.
- **Ecosystem.** NumPy subset mapped to Eigen or xtensor, calls from compiled code back into CPython for library calls, a VS Code extension and a CI mode that fails a build when a converted function stops matching.
- **Retargeting.** The IR is language-agnostic past the front end, so a second front end (or the WebAssembly back end dropped from this cycle) is an addition, not a rewrite.

13.3 Publication potential

The Semantic Fidelity Report, a positioned, machine-generated account of where a translation could differ, treated as a first-class output of a source-to-source compiler and paired with replay verification on recorded production calls, is the part we believe is publishable. To make it a paper rather than a project the team would need a larger, independent corpus than ours, a comparison against Cython, mypyc and Codon on the same programs, and a proper study of the false-accept rate of replay verification (for example by mutation testing the compiler).

14. References

1. Aho, A. V., Lam, M. S., Sethi, R. and Ullman, J. D. (2006). *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley.

2. Pierce, B. C. and Turner, D. N. (2000). Local type inference. *ACM Transactions on Programming Languages and Systems*, 22(1), 1-44.

3. Pratt, V. R. (1973). Top down operator precedence. *Proc. 1st ACM Symposium on Principles of Programming Languages*, 41-51.

4. Cousot, P. and Cousot, R. (1977). Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. *Proc. 4th ACM POPL*, 238-252.

5. McKeeman, W. M. (1998). Differential testing for software. *Digital Technical Journal*, 10(1), 100-107.

6. Yang, X., Chen, Y., Eide, E. and Regehr, J. (2011). Finding and understanding bugs in C compilers (Csmith). *Proc. PLDI*, 283-294.

7. Pnueli, A., Siegel, M. and Singerman, E. (1998). Translation validation. *Proc. TACAS*, LNCS 1384, 151-166.

8. Leroy, X. (2009). Formal verification of a realistic compiler. *Communications of the ACM*, 52(7), 107-115.

9. ISO/IEC (2017). *ISO/IEC 14882:2017, Programming languages: C++*.

Official documentation consulted for the comparison (accessed October 2026).

10. Cython documentation, "Cython - an overview". https://cython.readthedocs.io/en/latest/src/quickstart/overview.html

11. Numba documentation, "Overview" and "Glossary" (nopython mode, type inference). https://numba.readthedocs.io/en/stable/

12. PyPy, project home page (speed and compatibility statements). https://pypy.org/

13. mypyc documentation, "Introduction". https://mypyc.readthedocs.io/en/latest/introduction.html

14. Nuitka project, README and user manual. https://github.com/Nuitka/Nuitka , https://nuitka.net/doc/user-manual.html

15. Codon, README and documentation (Apache-2.0). https://github.com/exaloop/codon , https://docs.exaloop.io/

16. Pythran documentation. https://pythran.readthedocs.io/en/latest/

17. Shed Skin, README. https://github.com/shedskin/shedskin

18. LPython, project page (alpha status, backends). https://lpython.org/

19. Python documentation, `sys.monitoring` and `sys.setprofile`. https://docs.python.org/3/library/sys.monitoring.html

Reused open-source components.

20. pybind11 (BSD-3-Clause), seamless operability between C++11 and Python. https://github.com/pybind/pybind11

21. FastAPI (MIT), Uvicorn (BSD-3-Clause), Starlette (BSD-3-Clause), Pydantic (MIT), websockets (BSD-3-Clause).

22. React 18 (MIT), Vite 5 (MIT), TypeScript 5 (Apache-2.0), Tailwind CSS 3 (MIT), Zustand (MIT), Framer Motion (MIT), cmdk (MIT), xterm.js (MIT).

23. Monaco Editor and @monaco-editor/react (MIT). Inter and JetBrains Mono fonts (SIL OFL 1.1) via Fontsource.

24. Lucide icons, `lucide-react` 0.460.0 (ISC licence), used in the application and in the figures of this report.

25. MinGW-w64 GCC 16.2 (UCRT build) distributed by WinLibs (GPL with runtime-library exception). https://winlibs.com/

26. Inno Setup 6.7.3 (Jordan Russell and Martijn Laan; free licence). https://jrsoftware.org/isinfo.php

27. CPython 3.13 relocatable distribution via the NuGet `python` package (PSF licence).

28. Playwright for Python (Apache-2.0) and Matplotlib (PSF-based licence), used to produce this report's screenshots and charts; python-docx (MIT) to generate the .docx.

AI coding assistants. PYTHIA was developed with the help of AI coding assistants. The repository history records Anthropic Claude (Opus 5.5 and Sonnet 5.5, used through Claude Code) and GitHub Copilot as co-authors in `Co-authored-by` trailers on 99 of 143 commits, including the dictionary-library pull request. This report and the viva guide were also drafted with such an assistant from the repository and from commands run on the team's machine. The team is responsible for the content.

Appendix A. User manual

A condensed version of `USER_GUIDE.md`. Screens are in Appendix H.

1. **Install.** Run `PYTHIA-Studio-Setup-1.1.0.exe` (per-user, no administrator rights). Start **PYTHIA Studio** from the Start menu. Python, the C++ compiler and the web UI are inside the installer.
2. **First start.** A three-step setup checks the computer (Python, compiler, headers, data folder), offers an optional AI key (skip it: the in-house path needs none), and then offers *Replay sample run*, *Open the demo project* or the *Playground*.
3. **Bring code in.** *Open folder*, *Clone repository* (a public GitHub URL), drop a `.py` file, or use the Playground. Give an **entry command** such as `python main.py` so PYTHIA can trace the program, rank functions by measured time and record real calls.
4. **Watch the pipeline.** The stepper runs *Ingest, Scan, Trace, Rank, Fix, Compile, Verify, Benchmark, Splice, Export*. Click any stage for its raw output; the agent panel narrates each step.
5. **Review.** Each function has a status chip (*Verified, Guarded, Fixable, Declined, Failed, Not eligible*). Open one for the **Diff** (original against accelerated file), the **three-way view** (Python, PySub, C++ with the fidelity gutter) or the **Journey** (sixteen steps from original source to splice).
6. **Accept and export.** Accept or reject per function (or accept all verified ones), then **Export** a folder or zip: your project with `_pythia_accel/`, `PYTHIA_REPORT.md` and `build_accel.sh`. Your original folder is never modified.
7. **Playground.** Type or paste a program; the C++ appears as you type. *Convert* turns plain Python into PySub with the in-house converter, and *Run* executes the original under CPython and the converted program compiled to C++ on the same input, reporting whether the output is identical.
8. **Command line.** `python -m pythia.cli prog.py --run` (compile and run), `--fidelity` (divergence report), `--emit-only` (print C++), `--mode leetcode` (a solution class without `main`), `--lib f,g` (build an importable module).

Appendix B. Installation guide

Table B.1. Installation paths.

You have	Do this
Windows 10 or 11	Run `PYTHIA-Studio-Setup-1.1.0.exe` from `dist/` (or the GitHub release); or unzip `PYTHIA-Studio-1.1.0-portable.zip` and run the launcher. Nothing else to install.
Linux, macOS, WSL	`./studio.sh` from a checkout, or the one-line installer `scripts/install.sh`, then `pythia-studio --app`. Needs Python 3.10 or newer, a C++17 compiler and `pip install pybind11`.
Docker	`docker build -t pythia-studio .` and `docker run --rm -p 127.0.0.1:8765:8765 -v pythia-data:/data pythia-studio`
Developers	`python -m pythia_studio --no-open --port 8797` (set `PYTHIA_HOME` to isolate state). UI development: `cd studio/web && npm install && npm run build` (the built `dist/` is committed).
Build the installer	`python packaging/windows/build.py` downloads the pinned CPython, WinLibs GCC and Inno Setup, assembles the tree and writes `dist/PYTHIA-Studio-Setup-1.1.0.exe`.

Troubleshooting. `pythia-studio --doctor` prints each check with its fix. If errors look impossible, check for a stale server: the doctor reports a build fingerprint and the launcher refuses a server with a different one; on this team's machines the usual culprit was an old server left running inside WSL and answering on the same port.

Appendix C. Sample runs

Per-function results of the Studio runs in Table Table 10.3, zero-LLM path. `Speedup` is the median over the replayed calls; `replay` is matched calls over replayed calls; `declined` rows show the measured probe speedup and the reason.

textlab

Table C.1. Functions compiled in `textlab`.

Function	File	Status	Speedup	Share	Replay	Note
`stage_sim`	main.py	guarded	4.7x	30.1%	20/?	
`monte_carlo_pi`	montecarlo.py	guarded	23.3x	12.5%	49/?	
`simulate_energy`	nbody.py	verified	10.6x	11.7%	49/?	
`edit_distance`	distance.py	guarded	8.1x	7.9%	500/?	
`prime_square_sum`	primes.py	guarded	20.7x	7.7%	50/?	
`assign_clusters`	kmeans.py	verified	35.0x	6.6%	80/?	
`collatz_longest`	collatz.py	guarded	40.8x	6.1%	49/?	
`life_generations`	life.py	declined	0.8x	5.9%	50/?	Compiled version measured 0.82x (slower than Python): marshalling cost outweighs
`is_prime`	primes.py	guarded	2.2x	5.3%	500/?	
`window_energy`	window.py	declined	0.3x	5.0%		Declined: marshalling ~3.668s (170520 calls x ~5003 elements) exceeds the ~1.163
`count_pairs`	bpe.py	guarded	1.3x	2.4%	80/?	

project-euler (problem 14)

Table C.2. Functions compiled in `project-euler (problem 14)`.

Function	File	Status	Speedup	Share	Replay	Note
`problem_014`	problem_014.py	declined	0.8x	98.3%	49/?	Compiled version measured 0.81x (slower than Python): marshalling cost outweighs
`collatz_sequence_len`	utils.py	guarded	21.3x	84.8%	500/?	
`problem_001`	problem_001.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_003`	problem_003.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_006`	problem_006.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_009`	problem_009.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_015`	problem_015.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_016`	problem_016.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`drop_spaces_or_hyphens`	problem_017.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_020`	problem_020.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`find_upper_bound`	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`problem_030`	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`find_upper_bound`	problem_032.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
`factorial`	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

Function	File	Status	Speedup	Share	Replay	Note
gcd	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_leap_year	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_palindromic	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_pandigital	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_prime	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
lcm	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digit_powers_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digits_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

project-euler (problem 10)

Table C.3. Functions compiled in `project-euler (problem 10)`.

Function	File	Status	Speedup	Share	Replay	Note
is_prime	utils.py	guarded	1.8x	72.3%	500/?	
problem_001	problem_001.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_003	problem_003.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_006	problem_006.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_009	problem_009.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_014	problem_014.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_015	problem_015.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_016	problem_016.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
drop_spaces_or_hyphens	problem_017.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_020	problem_020.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
find_upper_bound	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_030	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
find_upper_bound	problem_032.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
collatz_sequence_len	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
factorial	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
gcd	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_leap_year	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_palindromic	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_pandigital	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

Function	File	Status	Speedup	Share	Replay	Note
lcm	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digit_powers_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digits_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

project-euler (problem 30)

Table C.4. Functions compiled in `project-euler (problem 30)`.

Function	File	Status	Speedup	Share	Replay	Note
problem_030	problem_030.py	failed	n/a	99.3%	12/?	Replay mismatch (exception) on args -18: python raises ZeroDivisionError, compil
sum_of_digit_powers_in_number	utils.py	guarded	1.3x	52.4%	500/?	
find_upper_bound	problem_030.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (1 calls); the saving would be negligible, so
problem_001	problem_001.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_003	problem_003.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_006	problem_006.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_009	problem_009.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_014	problem_014.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_015	problem_015.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_016	problem_016.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
drop_spaces_or_hyphens	problem_017.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
problem_020	problem_020.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
find_upper_bound	problem_032.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
collatz_sequence_len	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
factorial	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
gcd	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_leap_year	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_palindromic	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_pandigital	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
is_prime	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
lcm	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so
sum_of_digits_in_number	utils.py	declined	n/a	0.0%		Cold: 0.00% of the traced runtime (0 calls); the saving would be negligible, so

textlab (before print fix)

Table C.5. Functions compiled in `textlab (before print fix)`.

Function	File	Status	Speedup	Share	Replay	Note
`stage_sim`	main.py	guarded	4.1x	31.5%	20/?	
`monte_carlo_pi`	montecarlo.py	guarded	29.2x	14.2%	49/?	
`simulate_energy`	nbody.py	verified	10.7x	11.5%	49/?	
`edit_distance`	distance.py	guarded	9.8x	8.0%	500/?	
`prime_square_sum`	primes.py	guarded	21.9x	6.9%	50/?	
`assign_clusters`	kmeans.py	verified	26.8x	6.7%	80/?	
`collatz_longest`	collatz.py	guarded	31.3x	6.3%	49/?	
`life_generations`	life.py	declined	0.7x	5.9%	50/?	Compiled version measured 0.72x (slower than Python): marshalling cost outweighs
`window_energy`	window.py	declined	0.3x	5.0%		Declined: marshalling ~3.668s (170520 calls x ~5003 elements) exceeds the ~1.178
`is_prime`	primes.py	guarded	2.4x	4.8%	500/?	
`count_pairs`	bpe.py	guarded	1.3x	2.3%	80/?	

Appendix D. Complete test table

All 78 positive and 88 negative oracle programs from the run logged in Appendix F. Time columns are single-run, whole-process times and include process start-up.

Table D.1. Positive corpus (`tests/positive`).

Positive program	CPython	PYTHIA	Ratio	Result
t01_arith.py	96.2 ms	8.0 ms	12.0x	PASS
t02_lists.py	97.0 ms	9.5 ms	10.3x	PASS
t03_strings.py	100.7 ms	9.8 ms	10.3x	PASS
t04_control.py	113.3 ms	8.2 ms	13.7x	PASS
t05_floats.py	91.7 ms	9.0 ms	10.2x	PASS
t06_boolops.py	119.3 ms	9.0 ms	13.2x	PASS
t07_bubble.py	103.3 ms	6.8 ms	15.2x	PASS
t08_nested.py	87.5 ms	8.0 ms	10.9x	PASS
t09_sieve.py	109.9 ms	10.2 ms	10.8x	PASS
t100_linked_list_inferred.py	188.6 ms	7.3 ms	25.8x	PASS
t101_inheritance_dispatch.py	76.8 ms	7.6 ms	10.1x	PASS
t102_static_class_property.py	90.0 ms	8.5 ms	10.6x	PASS
t103_dunders_sorting.py	217.7 ms	11.6 ms	18.7x	PASS
t104_leetcode_solution.py	189.3 ms	9.0 ms	21.0x	PASS
t105_bst_stack_queue.py	214.3 ms	12.1 ms	17.7x	PASS
t106_dataclass_records.py	201.2 ms	14.1 ms	14.3x	PASS
t107_class_attrs_accounts.py	158.9 ms	10.9 ms	14.5x	PASS
t108_graph_tree_people.py	202.7 ms	12.5 ms	16.2x	PASS
t109_class_error_paths.py	242.5 ms	9.0 ms	27.0x	PASS
t10_collatz.py	830.2 ms	34.9 ms	23.8x	PASS
t110_private_inplace_membership.py	210.0 ms	12.5 ms	16.8x	PASS
t111_assert_and_nested_init.py	124.9 ms	8.7 ms	14.3x	PASS
t112_stdlib_heap_bisect_deque_copy_sys.py	495.2 ms	10.6 ms	46.9x	PASS
t113_stdlib_itertools_enum_setcomp.py	141.5 ms	10.6 ms	13.4x	PASS
t11_matmul.py	224.0 ms	14.4 ms	15.5x	PASS
t12_err_index.py	237.1 ms	12.3 ms	19.2x	PASS
t13_err_zerodiv.py	280.3 ms	11.6 ms	24.2x	PASS
t14_edges.py	140.7 ms	9.3 ms	15.2x	PASS
t15_classes_dict_nested_try.py	138.4 ms	7.8 ms	17.8x	PASS
t160_dict_get1.py	132.4 ms	7.6 ms	17.5x	PASS
t161_slice_assign_del.py	124.5 ms	7.8 ms	16.0x	PASS
t162_nlargest_str_container.py	125.8 ms	8.1 ms	15.4x	PASS
t163_try_else_finally.py	90.8 ms	7.3 ms	12.5x	PASS
t164_recursion_stdin.py	99.2 ms	16.3 ms	6.1x	PASS
t165_semicolons_inline.py	113.2 ms	8.2 ms	13.8x	PASS
t16_classes_methods_exceptions.py	121.6 ms	8.3 ms	14.7x	PASS
t17_loopvar_reassign_immune.py	104.4 ms	8.8 ms	11.9x	PASS
t18_pysub11_features.py	115.2 ms	12.6 ms	9.1x	PASS
t19_error_messages.py	129.3 ms	7.3 ms	17.6x	PASS
t20_ifexp_not.py	95.6 ms	9.3 ms	10.3x	PASS
t21_list_sorted_sources.py	120.7 ms	7.8 ms	15.5x	PASS
t22_ord_chr.py	210.6 ms	8.6 ms	24.5x	PASS
t23_chained_compare.py	124.2 ms	8.8 ms	14.2x	PASS
t24_tuples.py	118.2 ms	7.0 ms	17.0x	PASS
t40_list_dict_comprehensions.py	119.0 ms	9.1 ms	13.0x	PASS
t41_generator_consumers.py	112.2 ms	9.0 ms	12.5x	PASS

Positive program	CPython	PYTHIA	Ratio	Result
t42_enumerate_zip_reversed.py	120.3 ms	7.1 ms	17.1x	PASS
t43_key_functions.py	124.2 ms	11.2 ms	11.1x	PASS
t44_comprehension_edges.py	225.2 ms	12.6 ms	17.9x	PASS
t45_dict_changed_during_iteration.py	226.1 ms	8.2 ms	27.7x	PASS
t46_tuples_with_iteration.py	111.7 ms	7.9 ms	14.1x	PASS
t50_defaults_kwargs.py	150.8 ms	10.3 ms	14.7x	PASS
t51_optional_linked.py	224.5 ms	9.1 ms	24.8x	PASS
t52_isinstance.py	201.8 ms	6.9 ms	29.2x	PASS
t53_module_globals.py	223.6 ms	7.8 ms	28.7x	PASS
t54_sets.py	220.9 ms	7.4 ms	29.9x	PASS
t55_dict_values.py	215.3 ms	8.9 ms	24.2x	PASS
t56_optional_scalars.py	150.7 ms	10.8 ms	13.9x	PASS
t57_collections.py	244.3 ms	19.0 ms	12.8x	PASS
t70_bitops.py	150.0 ms	10.5 ms	14.2x	PASS
t71_math.py	135.4 ms	12.7 ms	10.7x	PASS
t72_slices.py	104.8 ms	8.1 ms	12.9x	PASS
t73_str_methods.py	122.2 ms	15.8 ms	7.7x	PASS
t74_list_dict_methods.py	103.8 ms	11.6 ms	8.9x	PASS
t75_pow_int_conv.py	110.6 ms	9.8 ms	11.3x	PASS
t76_format_percent.py	99.2 ms	8.1 ms	12.3x	PASS
t77_err_sqrt.py	228.0 ms	5.8 ms	39.0x	PASS
t78_err_dict_pop.py	188.9 ms	8.4 ms	22.5x	PASS
t79_err_negshift.py	182.0 ms	7.3 ms	25.0x	PASS
t80_err_pow_inverse.py	186.2 ms	7.8 ms	23.7x	PASS
t81_err_slice_step.py	251.1 ms	6.6 ms	38.3x	PASS
t82_err_log0.py	197.4 ms	8.7 ms	22.6x	PASS
t83_repr_concat.py	101.0 ms	7.7 ms	13.2x	PASS
t90_list_repeat.py	111.2 ms	7.9 ms	14.1x	PASS
t91_chained_assign.py	112.9 ms	7.5 ms	15.0x	PASS
t92_fstring_eq.py	116.8 ms	6.9 ms	17.0x	PASS
t93_cond_mixed_boolop.py	127.2 ms	8.0 ms	15.9x	PASS
t94_optional_tuple.py	147.7 ms	7.2 ms	20.6x	PASS

Table D.2. Negative corpus (`tests/negative`): the compiler must reject each with a diagnostic containing the expected text.

Negative program	Expected text	Result
n01_rebind.py	cannot assign	REJECTED
n02_badarg.py	expected int, found str	REJECTED
n03_undefined.py	is not defined	REJECTED
n04_return_type.py	returning str from a function declared -> int	REJECTED
n05_mixed_list.py	incompatible element types	REJECTED
n06_missing_return.py	without returning a value	REJECTED
n07_empty_list_infer.py	cannot infer element type	REJECTED
n08_arity.py	expects 2 argument	REJECTED
n09_bad_index.py	index must be int	REJECTED
n100_print_subclass_no_repr.py	defines neither __str__ nor __repr__	REJECTED
n101_multiple_inheritance.py	multiple inheritance	REJECTED
n102_override_signature.py	different signature	REJECTED
n103_classmethod_via_subclass.py	called through subclass	REJECTED
n104_field_not_inferable.py	cannot infer the type of field 'items'	REJECTED
n105_sort_without_lt.py	'<' not supported between instances of 'P' and 'P'	REJECTED
n106_operator_not_defined.py	unsupported operand type(s) for +	REJECTED

Negative program	Expected text	Result
n107_property_no_setter.py	has no setter	REJECTED
n108_frozen_dataclass.py	frozen dataclass	REJECTED
n109_base_defined_later.py	must be defined before	REJECTED
n10_str_item_assign.py	does not support item assignment	REJECTED
n110_eq_in_hierarchy.py	cannot define __eq__	REJECTED
n111_iter_dunder.py	__iter__ is outside the PySub subset	REJECTED
n112_exception_subclass.py	may only inherit from another class	REJECTED
n113_class_attr_call.py	must be a constant	REJECTED
n114_dataclass_mutable_default.py	mutable default	REJECTED
n115_method_only_in_subclass.py	has no method 'bark'	REJECTED
n116_unknown_decorator.py	decorator @functools.lru_cache	REJECTED
n117_super_outside_method.py	super() is supported only	REJECTED
n118_instance_hides_class_attr.py	hides class attribute	REJECTED
n119_default_repr_in_fstring.py	defines neither __str__ nor __repr__	REJECTED
n11_compare_types.py	cannot compare	REJECTED
n12_break_outside.py	outside loop	REJECTED
n13_class_no_init.py	no __init__	REJECTED
n14_missing_init_fields.py	no __init__	REJECTED
n15_bad_field.py	no field	REJECTED
n16_dict_bad_key.py	dict key type	REJECTED
n17_unknown_exception.py	unknown exception type	REJECTED
n18_print_class.py	no reproducible repr	REJECTED
n19_nested_list_too_deep.py	deeper than two levels	REJECTED
n20_break_in_try_no_loop.py	outside loop	REJECTED
n21_ifexp_mixed.py	single static result type	REJECTED
n22_chain_impure_middle.py	chained comparison with a computed middle operand	REJECTED
n23_tuple_var_index.py	tuple index must be an integer constant	REJECTED
n40_lambda_value.py	lambda is supported only as the key= argument	REJECTED
n41_len_generator.py	generator expressions are supported only as the sole argument	REJECTED
n42_enumerate_len.py	enumerate() is supported only as the iterable	REJECTED
n43_unpack_list.py	cannot unpack	REJECTED
n44_comp_var_scope.py	name 'x' is not defined	REJECTED
n45_set_comprehension.py	set comprehensions are outside the PySub subset	REJECTED
n46_range_value.py	range() is supported only as the iterable	REJECTED
n47_list_key.py	a key function must return int, float, bool, str or a tuple of those	REJECTED
n48_bare_genexp.py	Generator expression must be parenthesized	REJECTED
n49_comp_unpack_list.py	cannot unpack a list[int] into 2 names	REJECTED
n50_unknown_kwarg.py	unexpected keyword argument	REJECTED
n51_duplicate_kwarg.py	multiple values for argument	REJECTED
n52_missing_arg.py	missing	REJECTED
n53_nonconst_default.py	default values must be constants	REJECTED
n54_optional_arith.py	may be None here	REJECTED
n55_isinstance_float.py	cannot be decided statically	REJECTED
n56_print_set.py	hash order	REJECTED
n57_iterate_set.py	iterate over sorted	REJECTED
n58_import_os.py	is not supported in PySub	REJECTED
n59_typing_not_imported.py	is not imported	REJECTED
n60_unbound_local.py	UnboundLocalError	REJECTED
n61_none_infer.py	cannot infer a type	REJECTED
n62_union.py	unions other than	REJECTED
n63_set_pop.py	set.pop()	REJECTED

Negative program	Expected text	Result
n64_is_int.py	implementation-defined	REJECTED
n65_int_default_float.py	CPython keeps the	REJECTED
n66_global_undefined.py	must be assigned at module level	REJECTED
n67_defaultdict_factory.py	makes values of type int	REJECTED
n68_optional_len.py	may be None here	REJECTED
n70_bitop_float.py	bitwise operators need int or bool	REJECTED
n71_unsupported_module.py	is not supported in PySub	REJECTED
n72_partition.py	returns a tuple	REJECTED
n73_dict_keys_print.py	live view	REJECTED
n74_math_unknown.py	has no attribute	REJECTED
n75_format_nonliteral.py	string-literal template	REJECTED
n76_slice_dict.py	does not support slicing	REJECTED
n77_pct_d_str.py	real number is required	REJECTED
n79_invert_float.py	unary	REJECTED
n80_hash.py	not defined	REJECTED
n81_gcd_float.py	needs an integer	REJECTED
n82_pct_few.py	not enough arguments	REJECTED
n83_format_attr.py	outside the PySub subset	REJECTED
n84_format_impure.py	must be used exactly once	REJECTED
n85_fstring_ascii.py	!a	REJECTED
n93_boolop_value_mixed.py	unrelated types	REJECTED

Appendix E. Grammar specification and API specification

Grammar. `docs/grammar.ebnf` is reproduced as committed. It specifies the PySub v0.1 core; Section 11.4 explains how the implemented language extends it.

```
(* PySub v0.1 -- the statically typed Python subset accepted by PYTHIA *)
(* BCSE307L Team 2 / Project A15.  Owner: Member 1 (Vishal Vivek).          *)

module      = { NEWLINE | funcdef | statement } EOF ;

funcdef     = "def" NAME "(" [ params ] ")" [ "->" type ] ":" block ;
params      = param { "," param } ;
param       = NAME ":" type ;
type        = "int" | "float" | "bool" | "str" | "None"
            | "list" "[" type "]" ;

block       = NEWLINE INDENT statement { statement } DEDENT ;

statement   = if_stmt | while_stmt | for_stmt | simple_stmt
            | "return" [ expr ] NEWLINE
            | "pass" NEWLINE | "break" NEWLINE | "continue" NEWLINE ;

if_stmt     = "if" expr ":" block { "elif" expr ":" block } [ "else" ":" block ] ;
while_stmt  = "while" expr ":" block ;
for_stmt    = "for" NAME "in" ( range_call | expr ) ":" block ;
range_call  = "range" "(" expr [ "," expr [ "," expr ] ] ")" ;

simple_stmt = annassign | assign | augassign | exprstmt ;
annassign   = NAME ":" type "=" expr NEWLINE ;
assign      = target "=" expr NEWLINE ;
augassign   = target augop expr NEWLINE ;
exprstmt    = expr NEWLINE ;
target      = NAME | postfix "[" expr "]" ;
augop       = "+=" | "-=" | "*=" | "/=" | "//=" | "%=" | "**=" ;

(* Expressions are parsed by precedence climbing (Pratt).                   *)
(* Binding powers, loosest to tightest:                                     *)
(*   or(1) < and(2) < comparison(4) < +,-(6) < *,/,//,%(8) < **(12, right)  *)
expr        = or_expr ;
or_expr     = and_expr { "or" and_expr } ;
and_expr    = comparison { "and" comparison } ;
comparison  = sum [ ( "==" | "!=" | "<" | "<=" | ">" | ">=" ) sum ] ;
sum         = term { ( "+" | "-" ) term } ;
term        = power { ( "*" | "/" | "//" | "%" ) power } ;
power       = unary [ "**" power ] ;                    (* right associative *)
unary       = ( "-" | "+" | "not" ) unary | postfix ;
postfix     = atom { "[" expr "]" | "." NAME "(" [ arglist ] ")" } ;
atom        = INT | FLOAT | STRING | "True" | "False" | "None"
            | NAME [ "(" [ arglist ] ")" ]
            | "(" expr ")"
            | "[" [ expr { "," expr } ] "]" ;
arglist     = expr { "," expr } ;

(* Lexical layer.  INDENT / DEDENT are synthesised by the scanner from      *)
(* leading whitespace (the off-side rule); blank and comment-only lines     *)
(* carry no indentation meaning.  Implicit line joining applies inside      *)
(* ( ) and [ ].                                                            *)
NAME        = ( letter | "_" ) { letter | digit | "_" } ;
INT         = digit { digit | "_" } ;
FLOAT       = INT "." { digit } [ exponent ] | INT exponent ;
STRING      = "'" { char } "'" | '"' { char } '"' ;
```

Server API (`pythia/studio/server/app.py`). All routes are served on 127.0.0.1 and reject foreign `Host` headers.

Table E.1. REST and WebSocket interface of the Studio server.

Method and path	Purpose
`GET /api/env`	compiler, pybind11, Python and LLM availability
`GET /api/demos`	bundled demo projects
`GET /api/settings`	read or change settings (`PUT` through the same path)
`POST /api/settings/test-key`	check an optional API key with a read-only call
`GET /api/doctor`	environment checks and build fingerprint
`POST /api/jobs`	list jobs; `POST` starts one (input: snippet, file, folder, git URL or demo)
`POST /api/upload`	upload a file or snippet

Method and path	Purpose
`GET /api/jobs`	list jobs; `POST` starts one (input: snippet, file, folder, git URL or demo)
`GET /api/jobs/{job}`	job state and summary
`GET /api/jobs/{job}/events`	events after a sequence number
`WS /api/jobs/{job}/ws`	WebSocket event stream (live or replay)
`POST /api/jobs/{job}/cancel`	cancel a running job
`GET /api/jobs/{job}/functions`	function summaries
`GET /api/jobs/{job}/function`	one function record, optionally with stage dumps
`POST /api/jobs/{job}/decision`	accept or reject a function
`GET /api/jobs/{job}/file-diff`	original against accelerated file
`POST /api/jobs/{job}/export`	write the accelerated project
`GET /api/jobs/{job}/export.zip`	download it as a zip
`GET /api/jobs/{job}/report`	printable summary
`GET /api/workspaces/recent`	recent workspaces
`GET /api/workspaces/{ws}/tree`	file tree of a workspace
`GET /api/workspaces/{ws}/file`	file contents
`GET /api/fs/list`	browse local folders
`POST /api/playground/compile`	PySub to C++ with diagnostics and fidelity notes
`POST /api/playground/run`	run original under CPython and the compiled version
`POST /api/playground/convert`	in-house conversion of plain Python to PySub
`GET /api/playground/convert/status`	progress of a conversion
`POST /api/stages`	tokens, AST, typed AST, PIR, C++ for a snippet

Appendix F. Evidence register

Every number in the report is traceable to a row here. Commands were run from the repository root at commit `95bbb1b` on the machine described in Table Table 5.1; raw logs are in `docs/review3/src/evidence/`.

Table F.1. Evidence register.

#	Claim	Command	Result	Date
1	Oracle: compiled output equals CPython on every positive program	`wsl .venv/bin/python tools/oracle.py -v`	78/78	2026-10-24
2	Oracle: invalid programs rejected with the expected text	same	88/88	2026-10-24
3	Oracle single-run median speedup	same	15.2x (min 6.1x, max 46.9x)	2026-10-24
4	Fidelity notes over positive corpus	`python docs/review3/src/fidelity_stats.py`	1280 notes: exact 545, guarded 726, divergent 9	2026-10-24
5	Untyped basics, no LLM	`python scripts/check_basics.py --jobs 6`	91/91	2026-10-24
6	Snippets	`python scripts/check_snippets.py`	21/22; refused: coin_change	2026-10-24
7	`coin_change` regression bisected	`git worktree add` at `6423a29` and `210a599`, `check_snippets.py coin_change`	passes at `6423a29`, refused from `210a599`; old conversion prints `3.0 -1.0`, CPython `3 -1`	2026-10-24
8	Library mode matches CPython	`wsl .venv/bin/python scripts/check_libmode.py`	OK: 74 calls, 22 class calls; cold build 15.05 s, cached 27 ms	2026-10-24
9	Generated programs	`tools/fuzz.py --n 150 --seed 0` then `tools/fuzz_run.py`	150/150 identical, 0 mismatches	2026-10-24
10	Property tests	`python scripts/property_test.py`	50/50 identical	2026-10-24
11	Kernel benchmark and guard elision	`wsl .venv/bin/python docs/review3/src/bench_kernels.py`	median 10.8x, geometric mean 6.6x; outputs identical: True	2026-10-24
12	Edge cases, boundary cost, memory over many calls	`wsl .venv/bin/python docs/review3/src/edge_cases.py`	14 cases; RSS 28640 to 28640 KB over 200000 calls	2026-10-24
13	Studio runs, zero LLM	`python docs/review3/src/studio_runs.py ...` against a server started with empty `GROQ_API_KEY` and `ANTHROPIC_API_KEY`	5 runs; see Table Table 10.3	2026-10-24
14	Stress harness on public repositories	`python scripts/stress.py OWNER/REPO --limit N`	not run this session (cut)	2026-10-24
15	Lines of code, test counts, symbol line numbers	`python docs/review3/src/collect_facts.py`	38,834 lines in product code and tools (non-blank), 78/88 oracle programs	2026-10-24
16	Environment	`python docs/review3/src/collect_env.py`	Intel(R) Core(TM) Ultra 9 185H; Windows Python 3.13.16; WSL Python 3.14.4	2026-10-24
17	Commit counts and AI co-author trailers	`git shortlog -sn`; `git log --format=%B` for `Co-authored-by`	143 commits; 138 owner; 99 with an AI trailer	2026-10-24
18	Installer and zip sizes	`ls -l dist/`	88.8 MiB and 211.7 MiB (built 19 Oct; not rebuilt this session)	2026-10-24
19	Grammar rule count	regex over `docs/grammar.ebnf`	33 named rules	2026-10-24
20	No `settrace`, no Electron, no Hindley-Milner in the code	repository search (grep)	no matches in product code	2026-10-24
21	LLM is optional, not the default path	read `pipeline.fix_one`, `llm.make_client`	annotate, then deterministic inference, then LLM only if a key exists	2026-10-24
22	Competitor descriptions	official pages fetched: Shed Skin, Nuitka, PyPy, LPython, mypyc, Numba, Pythran	Figure 4.1 cells; Cython and Codon pages were not retrieved in full (Cython from general knowledge, Codon from a search snippet), so those two rows are lower confidence	2026-10-24
23	Emitted code listing	`python -m pythia.cli e.py --emit-only`	Listing 7.1	2026-10-24

Claims in the earlier draft that were corrected or removed.

Table F.2. Corrections to the teammate's draft.

Draft claim	Finding	Action
Register numbers 24BDS0102, 24BDS0118, 24BDS0127 and Vishal as 24BDS	Review 1 report: 24BCE2377, 24BDS0459, 24BDS0132, 24BDS0134.	Corrected on the cover and in Section 12.
"Hindley-Milner" type inference	`typecheck.py` is bidirectional local inference with one fixed type per variable; the converter uses a lattice fixpoint. No unification-based generalisation exists (searched the code).	Removed; Section 7.3 and 7.4 describe the real algorithms.
"Electron desktop shell"	The app is a FastAPI server plus a React page shown in an Edge/Chrome `--app` window by `launcher.py`; no Electron anywhere in the repository (searched).	Corrected in Sections 5 and 6.
"zero memory leaks"	Standalone binaries never free memory (D7). Library mode frees per call; measured RSS flat over 200000 calls.	Replaced by the measured statement.

Draft claim	Finding	Action
`sys.settrace`	The tracer uses `sys.monitoring` with a `sys.setprofile` fallback (`_tracer_boot.py:66`); `settrace` appears nowhere.	Corrected in Section 7.9.
`symbol_table.py`, `stages.py` ownership, `splice.py` in `pythia/`	`symbol_table.py` does not exist (scopes are in `typecheck.py`); `stages.py` exists but is a read-only stage dumper for the UI; `splice.py` is `pythia/studio/splice.py`.	File list rebuilt from the tree (Listing 8.1).
Runtime owned by Aaryan; `lower.py` and `optimize.py` plus `stages.py` for Jahnavi	The Review 1 matrix and file headers give the runtime and fidelity to Jahnavi; the emitter and oracle to Aaryan.	Section 12 follows the matrix and git.
Made-up commit messages ("feat(front): lexer indentation", ...)	`git log` has no such messages; the first commit is a single squash.	Replaced by the real commits in Appendix G.
Intel i7-13700H, Python 3.11, GCC 13.2	Measured environment is in Table Table 5.1 (Intel(R) Core(TM) Ultra 9 185H, Python 3.13.16 and 3.14.4, GCC 15.2.0 and 16.2).	Replaced.
Speedups 12.4x to 84.6x, geometric mean 28.7x; 8-row benchmark table	No such measurements exist in the repository or in our runs. Our kernels: 0.03x to 29.7x, geometric mean 6.59x.	Replaced by Table Table 10.2.
"Assembly confirms vectorization"; "100% bounds-check elision"	No assembly was inspected, and guards are elided only where provable (Section 10.2 gives counts).	Removed.
"6,000+ LOC Quanta"; "142 functions"	`demo/quanta` is about 3,800 lines (`DECISIONS.md`); function counts come from the Studio runs in Table Table 10.3.	Replaced.
"Zero-overhead" pybind11 splicing; "0% false positives"; "100% of divergences caught"	Marshalling copies data (measured); replay can miss unexercised paths.	Replaced by measured costs and by Section 11.
Figures 4.1, 6.1, 10.1 referenced but absent	Report now contains every figure it refers to; cross-references are generated.	Fixed.
Test-case table TC-01 to TC-10 and the emitted-code listing	The loop listing used names (`_t_ctr`) that the emitter does not produce, and TC-07 claimed assembly evidence.	Rebuilt from runs (Table Table 9.2, Listing 7.1).

Appendix G. Contribution log

From `git log` and `git shortlog` at the commit above. Merge commits are listed where they are the only evidence.

Table G.1. Commits by teammates and the main milestones.

Date	Commit	Author	What	Member (by Review 1 matrix)
2026-09-04	84f1251	Vishal Vivek	Inital commit	owner (first commit holds the Review 2 code base)
2026-09-04	9264cbb	AaryanGupta01	Migrate backend from C11 to C++17	Aaryan (M4)
2026-09-04	086b6f8	Vishal Vivek	Merge pull request #1 from AaryanGupta01/cpp17-backend	Aaryan (M4), merge of PR 1
2026-09-04	16450af	Copilot	Add PyDict: Python-compatible dictionary library for PYTHIA compiler	Rujuta (M2), authored by Copilot
2026-09-09	ddc975b	Jahnavi	Add Python-compatible set runtime library	Jahnavi (M3)
2026-09-09	079bcb5	Vishal Vivek	Merge pull request #2 from rujuta-student/rujuta-student-add-dict-library	Rujuta (M2), merge of PR 2
2026-09-09	4ef0102	Vishal Vivek	Merge pull request #3 from Jahnavikoliparthy/add-pyset-library	Jahnavi (M3), merge of PR 3
2026-09-10	533f875	Vishal Vivek	Final Cleanup Before Review 2	owner (Studio, proposed split in Section 12)
2026-10-17	2f46833	Vishal Vivek	Add PYTHIA Studio build prompt, contracts and decisions	owner (Studio, proposed split in Section 12)
2026-10-18	74bfa7d	Vishal Vivek	Run natively on Windows: toolchain discovery (bundled MinGW-w64), static .pyd builds, Windows process control	owner (Studio, proposed split in Section 12)
2026-10-18	cf2cfb8	Vishal Vivek	Playground: AI conversion of plain Python to PySub (type-checker loop), stdin for interactive programs, run or	owner (Studio, proposed split in Section 12)
2026-10-18	37acf6d	Vishal Vivek	Windows app: self-contained bundle (Python 3.13 + MinGW-w64 GCC 16), launcher exe, Inno Setup installer, first	owner (Studio, proposed split in Section 12)
2026-10-18	d344271	Vishal Vivek	WIP: in-house converter type lattice and inference engine	owner (Studio, proposed split in Section 12)
2026-10-19	0cad7e4	Vishal Vivek	fix(convert, emit, studio): resolve gaps for coin_change and group_anagrams, always enable in-house conversion	owner (Studio, proposed split in Section 12)
2026-10-19	8c0e1dc	Vishal Vivek	Basics gate: 90 untyped everyday programs + check_basics.py (in-house path, no LLM, compiled vs CPython)	owner (Studio, proposed split in Section 12)
2026-10-19	f14f559	Vishal Vivek	feat: add property-based differential test harness, leetcode mode, and modernize tooling	owner (Studio, proposed split in Section 12)
2026-10-19	e4a4804	Vishal Vivek	feat(studio): restore LLM conversion option and cleanup workspaces to working demos	owner (Studio, proposed split in Section 12)
2026-10-19	95bbb1b	Vishal Vivek	feat(packaging): bundle demos and generate portable zip and Windows installer	owner (Studio, proposed split in Section 12)

Appendix H. User-interface screenshots and additional diagrams

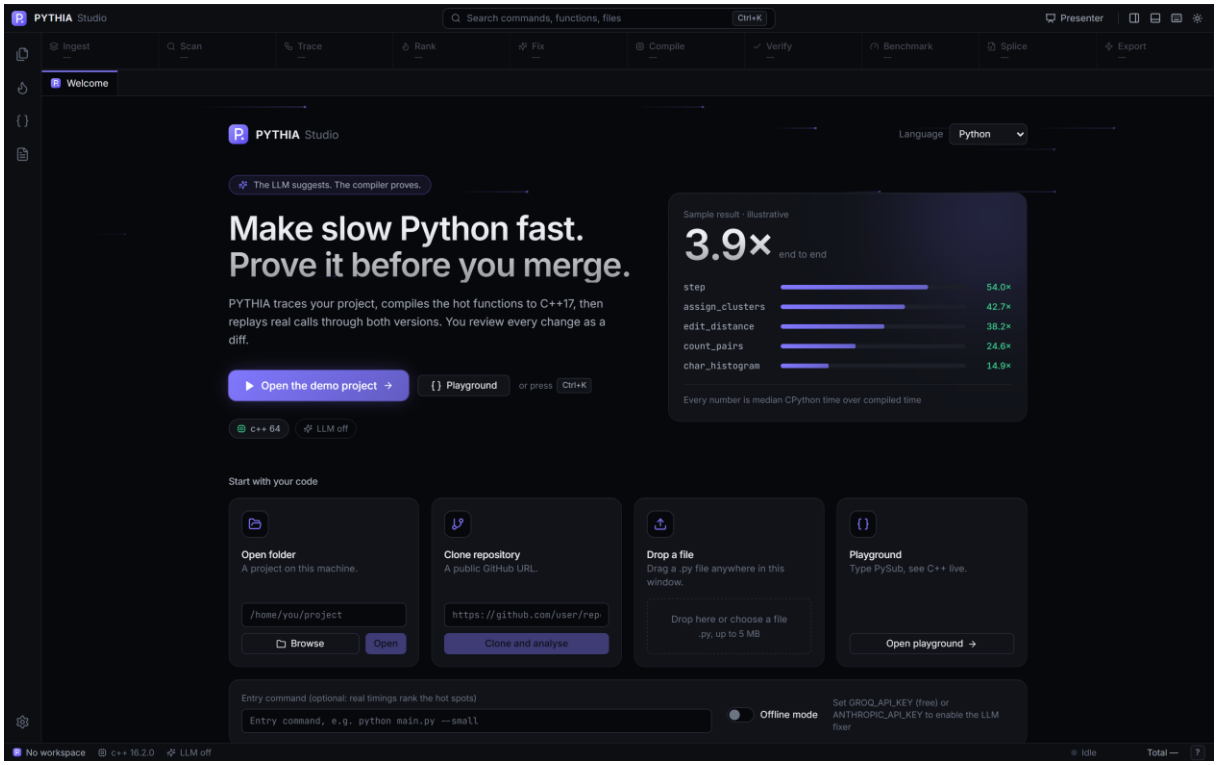


Figure H.1. Welcome screen: open a folder, clone a GitHub URL, drop a file, open the playground or replay a sample run.

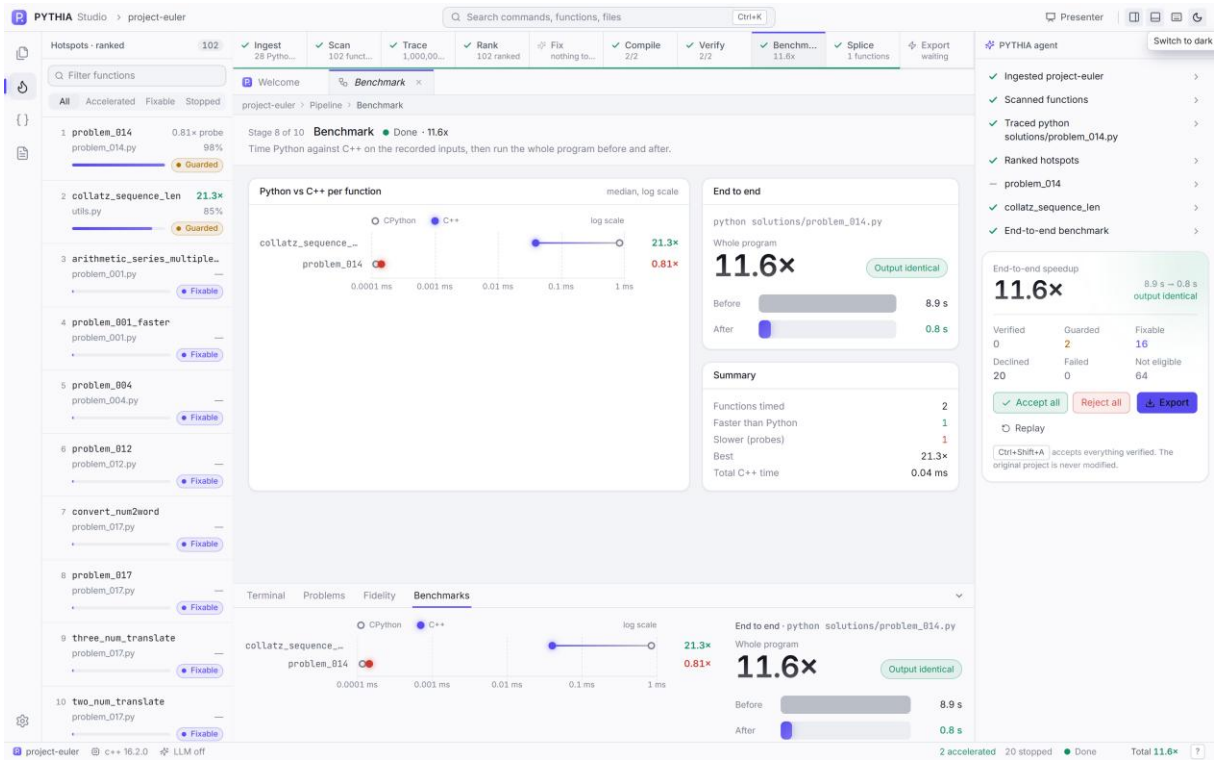


Figure H.2. A finished project run: the ten-stage pipeline stepper, ranked hotspots, agent narration and the end-to-end card.

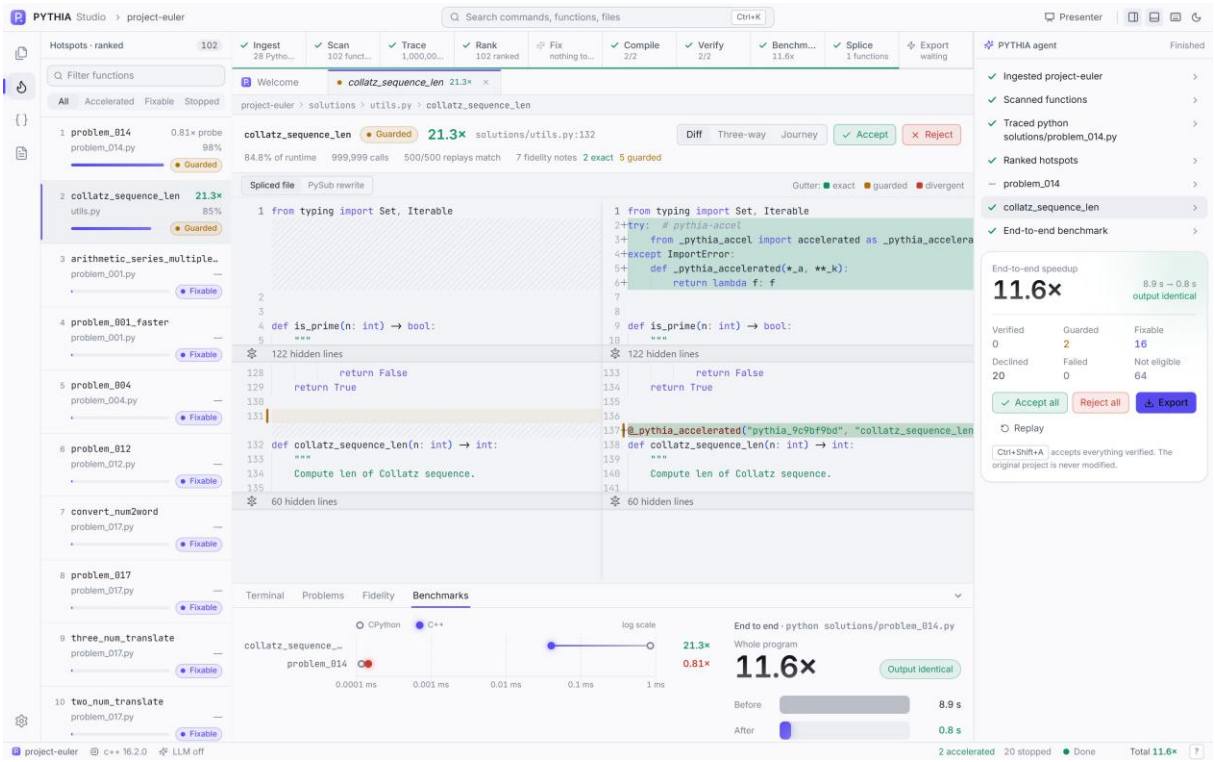


Figure H.3. Diff view with the fidelity gutter: one decorator line added above the original function, the body untouched.

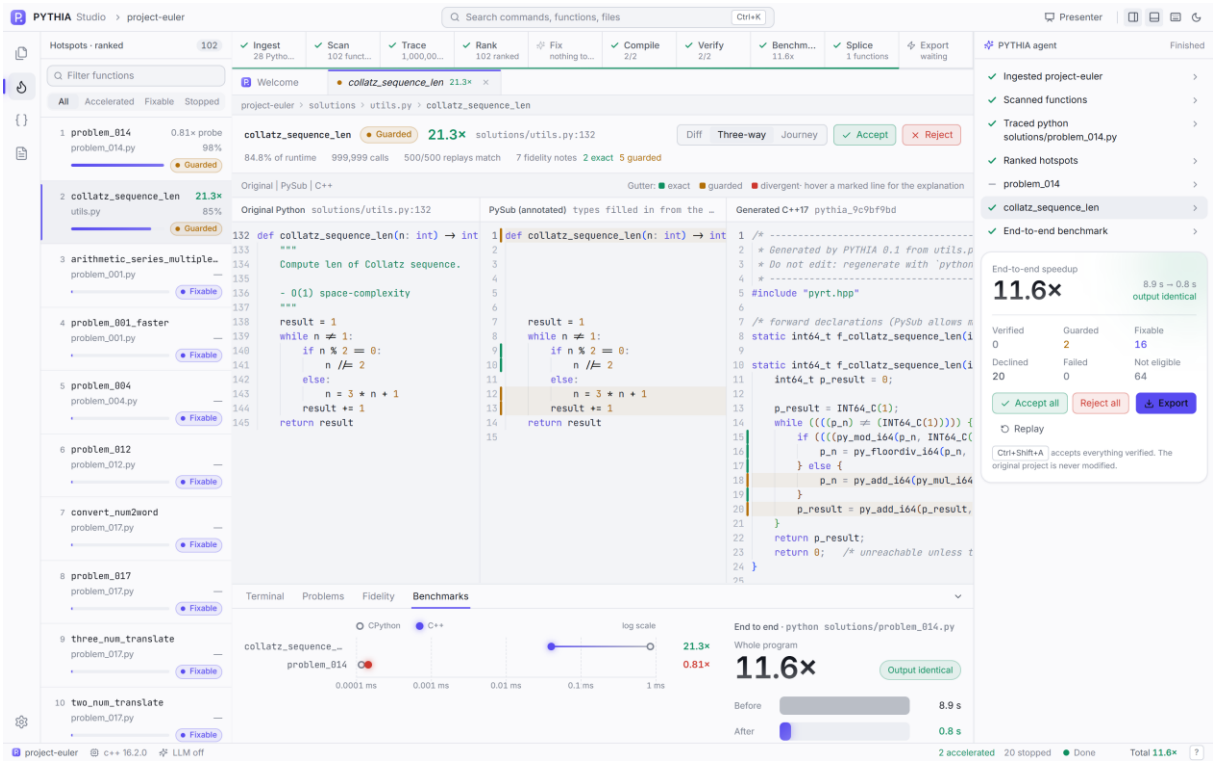


Figure H.4. Three-way view: Python, PySub and generated C++, with fidelity notes in the gutter.

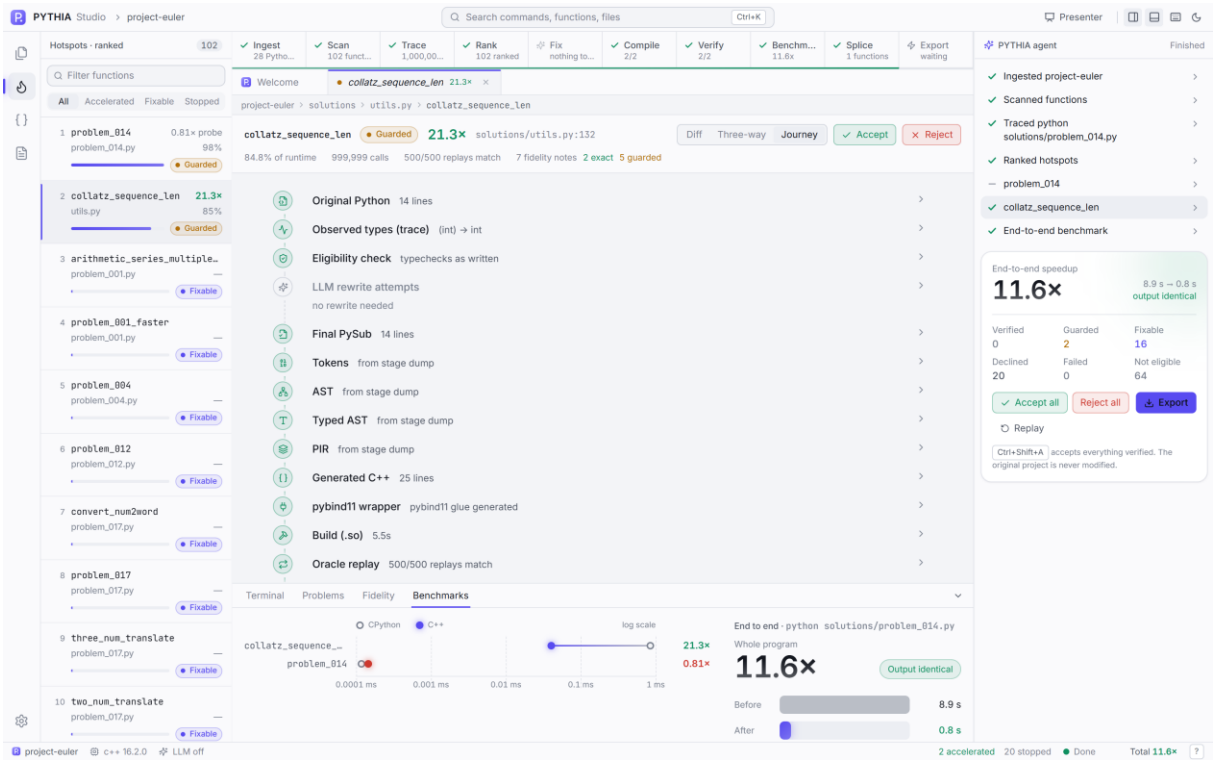


Figure H.5. The function journey: sixteen steps from original source to splice.

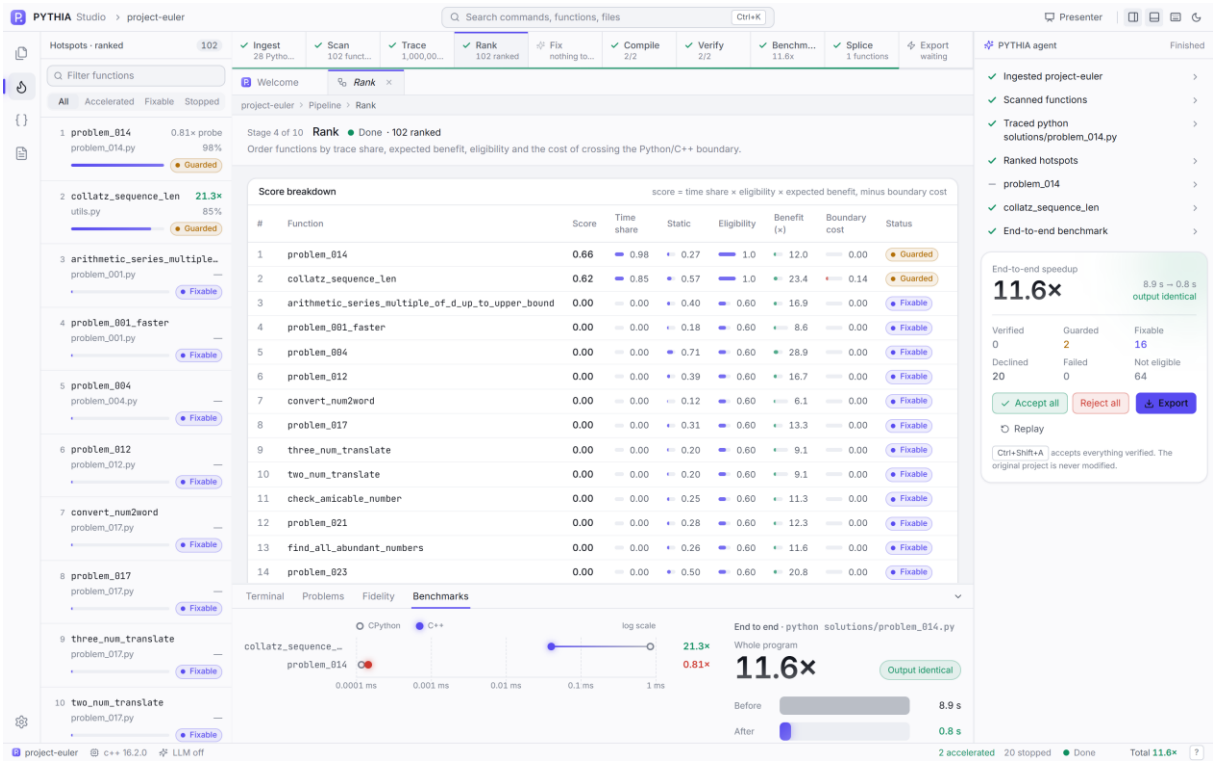


Figure H.6. Rank stage inspector: score breakdown per function.

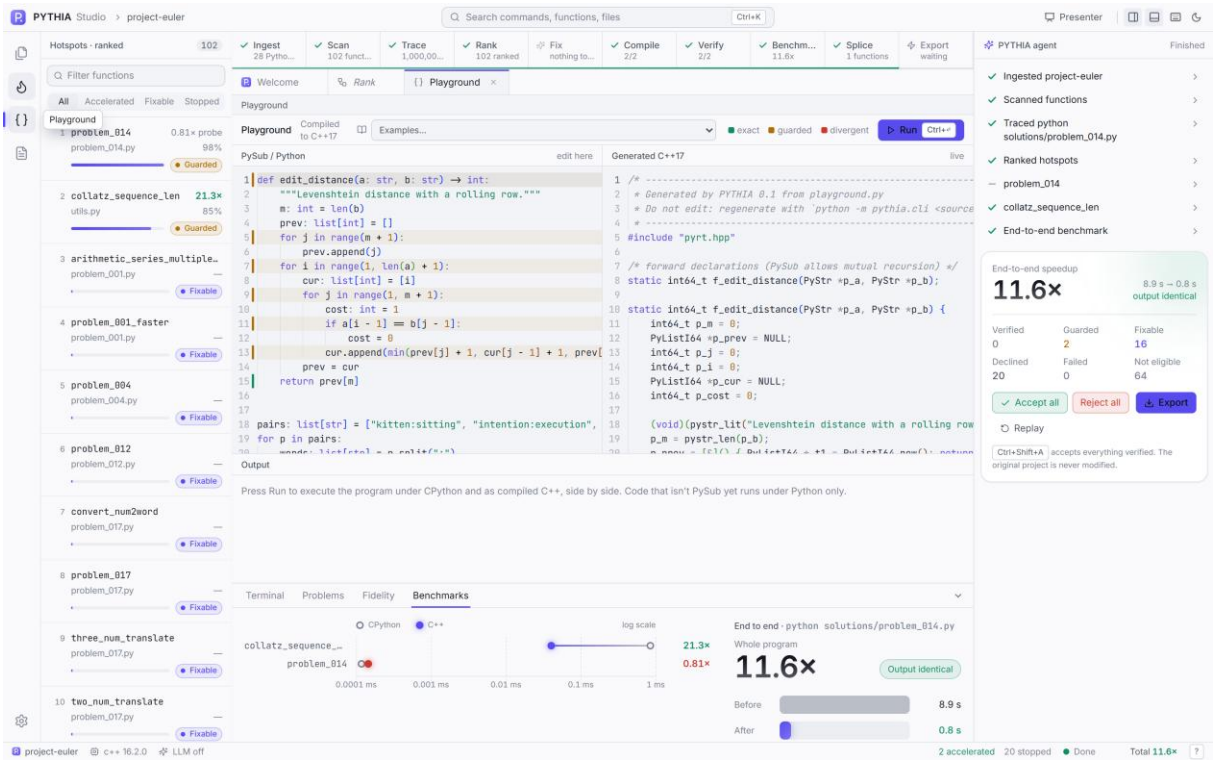


Figure H.7. Playground: convert, compile and compare against CPython.

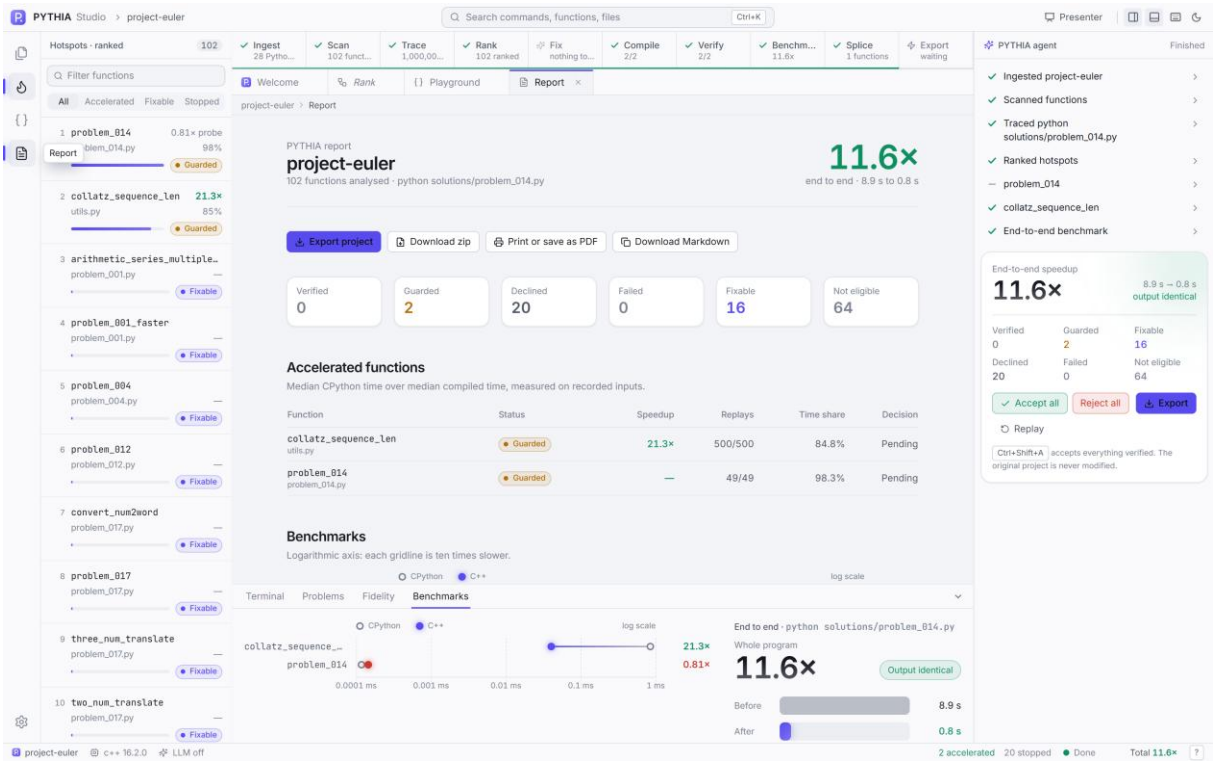


Figure H.8. Report page.